有向类型论中的纤维化

Benno Lossin

摘要

我们在 Riehl 和 Shulman 发展的综合**单纯类型论**中研究 ∞-范畴。具体而言，我们定义了**余笛卡尔纤维化**，并利用 **LARI 伴随**与**始截面**之间的一个新等价性，证明了它们的封闭性质。我们使用实验性证明辅助工具 rzk 对工作进行形式化，并将成果整合到 Riehl 等人的形式化工作中。除新工作外，本文还介绍了一般类型论、同伦类型论以及论文其余部分所使用的单纯类型论。

致谢

我要感谢我的导师 Tashi Walde，他就本论文的内容进行了大量讨论并付出了大量时间。他帮助我理解和深入这一理论，使我不必担心迷失方向。我还要感谢我的导师 Claudia Scheimbauer，感谢她将我介绍给 Tashi，并指导本论文的完成。

目录

第一章 引言 **4**

1.1 历史 4

1.2 与其他学科的联系 5

1.2.1 基础数学与逻辑学 5

1.2.2 形式化数学 6

1.2.3 计算机科学 7

1.2.4 构造性数学与证明相关性 7

1.2.5 同伦论 8

1.2.6 序理论、范畴论与高阶范畴论 8

1.3 向 HoTT 添加方向性的两种方法 8

1.3.1 用形状扩展 HoTT 8

1.3.2 限制 Π-类型 9

1.3.3 圣杯 9

1.4 动机 9

1.5 综合 vs 分析 10

1.6 论文结构 10

第二章 同伦类型论 **11**

2.1 类型论基础 11

2.1.1 推理规则与判断 12

2.1.2 规则与类型检查 15

2.1.3 类型论蕴涵 15

2.1.4 判断相等 17

2.1.5 一般句法 18

2.1.6 单位类型 18

2.1.7 空类型 19

2.1.8 Σ 类型 19

2.1.9 Π 类型 22
2.1.10 宇宙 23
2.1.11 同伦 25
2.1.12 可缩性 28
2.1.13 命题 29
2.1.14 等价 30
2.1.15 嵌入 32
2.1.16 幺半等价性 32
2.1.17 Grothendieck 构造 33
2.2 单纯 HoTT 与范畴论 34
2.2.1 立方体、拓扑和形状 34

第一章　引言

现代类型论将几个特殊的概念和思想汇集到一个基础数学理论中，取代了逻辑学和集合论。它与数学中许多基本且非常重要的学科相关联：高阶范畴论、同伦论、序理论、逻辑学、基础数学、形式化数学和构造性数学。它植根于基础数学、逻辑学和理论计算机科学，并通过其与形式化的良好兼容性提供了高度的正确性。此外，它还提供了最直观的数学诠释之一。与同伦论和高阶范畴的联系直到最近才被发现，这导致了对同伦类型论（HoTT）这一新分支的许多有趣探索，最终形成了无公理基础计划的工作 [1]。

本论文总结了由 E. Riehl 和 M. Shulman [2] 发展的单纯类型论，作为同伦类型论的扩展。E. Riehl 等人 [3] 使用由 N. Kudasov、A. Abounegm 和 D. Danko [4] 基于 Riehl 和 Shulman 理论创建的证明辅助工具 rzk 将该理论形式化。U. Buchholtz 和 J. Weinberger [5] 以及 C. Bardomiano Martinez [6] 对该理论的若干扩展也已被 E. Riehl 等人 [3] 部分形式化。我们通过在本论文中形式化余笛卡尔纤维化并将其贡献到 [3] 来继续他们的工作。我们特别遵循了 U. Buchholtz 和 J. Weinberger [5] 以及 J. Weinberger [7] 的工作，他们将余笛卡尔纤维化作为 LARI 族来研究。我们通过证明始截面与 LARI 伴随是等价的来将这些概念联系起来，这简化了形式化工作。我们还证明了余笛卡尔纤维化在乘积、拉回和复合下是封闭的。我们的形式化工作可以在线访问：https://github.com/BennoLossin/sHoTT，当它们被上游接受后，也将在 https://rzk-lang.github.io/sHoTT 以更易读的形式提供。

在本节的剩余部分，我们简要概述类型论中最重要的历史发展。然后我们讨论与其他数学分支的联系，并探讨综合方法的优点。我们还讨论了在同伦类型论中添加方向性的两种不同方式，以便研究高阶范畴。最后，我们概述本论文的结构。

1.1　历史

类型论起源于基础数学以及 1910 年 A. N. Whitehead 和 B. Russel [8] 在《数学原理》中对罗素悖论的解决。后来，A. Church [9] 在 1940 年发展了 λ 演算，作为一个“数学图灵机”，并引入类型以确保终止。此时，类型的概念已经具有与今天非常相似的含义。理论的另一个巨大飞跃是由 P. Martin-Löf 等人 [10] 在 1984 年通过引入直觉主义类型论实现的。他的开创性工作至今仍是许多类型论的基础，通常被称为 Martin-Löf 类型论（MLTT）。该理论倾向于构造性数学，并成为一个适当的基础系统。MLTT 被许多逻辑学家、数学家和理论计算机科学家进一步发展。我们关注的下一个里程碑是与同伦论的联系以及同伦类型论这一新分支的形成，该分支允许研究综合的 ∞-群胚。V. Voevodsky [11] 在 2010 年提出了幺半等

价性的思想，允许将弱等价的对象视为相等。这最终在 2013 年由无公理基础计划 [1] 出版了“HoTT 书”。该书全面概述了同伦类型论，并在其中推导出若干数学结果。该理论的许多进一步扩展已被探索。特别是，E. Riehl 和 M. Shulman [2] 在 2017 年通过一个严格的定向区间类型添加了方向性。这允许该理论以综合的方式定义 $(\infty, 1)$-范畴。

我们的工作核心基于两篇论文：U. Buchholtz 和 J. Weinberger [5] (2021) 以及 J. Weinberger [7] (2024)。他们在 Riehl 和 Shulman 的理论中从几个不同方向考虑了余笛卡尔纤维化的概念。

1.2　与其他学科的联系

数学是一种写下正确内容的句法游戏。理论的规则告诉我们哪些句法构造是有效的。类型论不仅描述了哪些句法构造（也称为项）是有效的，而且为每个项关联了一个类型。类型也是一种句法构造，但处于与项不同的层次。这个类型描述了该项所代表的值的“能力”。例如，函数 $f : A \to B$ 的类型告诉我们哪些值可以作为输入，以及可能得到哪些输出值。

数学家经常谈论值的“能力”，例如“这个函数是弱等价”或“这个对象在 XYZ 范畴中是始对象”。在类型论中，这些陈述无非就是该值（或一个派生值）具有某种类型。当数学家论证时，他们经常回忆并利用这些性质，例如“因为这个函数是等价”或“因为这个对象在 XYZ 中是始对象”。这对应于按照类型所规定的方式使用该值。

类型论自然地形式化了这种“能力”的概念，并使使用它们成为做数学的主要方式。类型论的核心信条是：

> “类型是命题，项是证明。”

这个主题是连接大多数其他学科与类型论的纽带。为命题给出证明归结为写出正确类型的项。跟踪不同类型的不同元素的能力是连接类型论与构造性数学和证明相关数学的纽带。一个类型的元素先验地可以是不同的，允许同一陈述有多个不同的证明。这给了我们与同伦论和高阶范畴的联系，因为我们可以使用项跟踪额外的结构，并且它们的非平凡相等性自然地出现在理论中。

1.2.1　基础数学与逻辑学

与之前许多基础系统相比，类型论是特殊的，因为它不是在逻辑学中表述的。类型论本身自然地包含了逻辑构造，并且直接使用逻辑学家定义逻辑的语言来表述。Curry-Howard 元理论同构描述了逻辑学和类型论之间的关系。表 1 简要概述了类型论如何整合逻辑学。我们将在第 2 节更详细地讨论这一点。与集合论相比，类型论更适合作为高阶范畴的基础系统。这是因为一个累积的、越来越大的宇宙层次是内置的，并且相等不是一个简单的“x 等于 y”的概念，而是包含了两个元素如何相等。此外，集合论没有内置的机制来跟踪高阶结构，而类型论则有。我们注意到，类型论不是一个单一的理论，而是一组可以用来构建理论的原则。就像数学中没有单一的逻辑学，而是有不同的变体一样。

逻辑概念	类型论概念
命题	类型
命题 P 的证明	类型 P 的项
真 $\top$	单元类型 $\mathbb{1}$
假 $\bot$	空类型 $\mathbb{0}$
合取 $A \wedge B$	乘积类型 $A \times B$
析取 $A \vee B$	不交并类型 $A + B$
蕴涵 $A \to B$	函数类型 $A \to B$
存在量词 $\exists x.P$	Σ-类型 $\Sigma(x : X, P)$
全称量词 $\forall x.P$	Π-类型 $(x : X) \to P$

表 1.1: 逻辑与类型理论之间的 Curry-Howard 元理论同构

1.2.2 形式化数学

类型论以非常特定的方式使用了逻辑学家的语言。类型论的规则以这样一种方式书写：每条规则都可以机械地应用，而不需要任何数学思想或选择。所谓的类型检查，规则仅决定哪些项属于哪个类型，哪些项无效。这使得它们非常适合推理和开发能够自动检查陈述正确性的程序。形式化数学增加了我们对陈述正确的信任度。此外，类型论方法将形式化数学置于中心位置，而在经典数学中，形式化往往是被事后考虑的。

大多数证明辅助工具建立在 P. Martin-Löf 等人 [10] 发展的 Martin-Löf 类型论之上，我们在第 1.1 节已经提到过。证明辅助工具有例如 Lean (L. de Moura, S. Kong, F. Doorn, and J. Raumer [12], L. d. Moura and S. Ullrich [13] and Lean Contributors [14])、Rocq (T. Coquand and G. Huet [15] and The Coq Development Team [16]) 以及 Isabelle/HOL (T. Nipkow and M. Wenzel [17] and Isabelle Contributors [18])。

对于 E. Riehl 和 M. Shulman [2] 的单纯类型论，有一个实验性的证明辅助工具，称为 rzk，由 N. Kudasov、A. Abouneggm 和 D. Danko [4] 创建。我们使用它来形式化本论文的大部分内容，其基础是 E. Riehl 等人 [3] 的工作。

证明辅助工具是发展良好的高阶范畴理论的非常有用的工具。这是因为高阶范畴是一个非常复杂的主题，在处理它们时通常需要记住大量数据。此外，在某些情况下，它们可能比普通范畴更不直观。证明辅助工具有助于避免这些陷阱并简化流程。

1.2.3 计算机科学

毫无疑问，类型论与计算机科学有着最强的联系之一。编程语言的研究——使用各种类型论——是计算机科学的一个完整学科，我们无法在这短短的一段中充分描述它。函数式编程语言基于 Church 的 λ 演算，许多现代语言都借鉴了它的概念。也有许多使用深奥类型论的实验性编程语言。计算和证明陈述通过 Curry-Howard 同构密不可分地联系在一起。

1.2.4 构造性数学与证明相关性

构造性数学关注于使用受限的基础系统，该系统不允许凭空产生值。例如，它们不假设选择公理。更令人惊讶的是，它们也不假设排中律，即 $A \vee \neg A$。在经典数学中，这是反证法中常用的不可或缺的工具。从构造性的角度拒绝这个公理的原因是，从公理 $A \vee \neg A$ 无法推导出两者中哪一个为真。这导致反证法是非构造性的。如果通过反证法证明了一个对象 x 的存在性，那么是通过证明它的不存在性会导致矛盾来实现的，从而从未真正构造出这个 x。

例 1.2.1. 展示这种行为的一个简单例子是以下陈述：

> 是否存在两个无理数 x 和 y 使得 x^y 是有理数?

答案是肯定的，我们可以使用排中律轻松证明。我们知道 $\sqrt{2}$ 是无理数。现在考虑 $x = y = \sqrt{2}$ 以及下面的论证。要么 $\sqrt{2}^{\sqrt{2}}$ 是无理数，要么是有理数。如果它是有理数，我们就完成了。否则，考虑 $x = \sqrt{2}^{\sqrt{2}}$ 和 $y = \sqrt{2}$，那么它们满足该陈述，因为：

$$x^y = \left(\sqrt{2}^{\sqrt{2}}\right)^{\sqrt{2}} = \sqrt{2}^2 = 2$$

从这个论证中，我们不清楚两种情况中哪一种成立，只知道其中一种成立。因此，构造主义者拒绝这种论证。

除了采纳构造性立场的哲学原因外，实际原因之一是任何证明都可以转化为产生定理结果的算法。这在计算机科学领域以及将数学应用于现实世界时特别有用。

类型论通过“类型作为命题”的原则邀请构造主义。我们通过必须创建的项来记录一个命题是如何被证明的，以展示一个类型的居留性。Σ-类型（类型论中对应存在量词 $\exists x.P(x)$ 的概念）通过构造该 Σ-类型的项来证明其居留性。这种构造需要两个元素：第一个是见证者 x，第二个是见证者满足条件 $P(x)$ 的证明。因此，如果给定一个存在量词的证明，我们总是可以获得一个具体的满足条件的元素。类似地，对于合取，我们需要提供两个输入陈述的证明元素来证明它们的合取。

这使得我们的理论不仅是构造性的，而且是证明相关的。我们关心一个命题可以被证明的方式。可能存在多种不同的方式。这也是同伦论的核心，它研究两个事物如何相等，以及如果它们相等，它们的相等性之间如何相互关联。

我们注意到，我们的理论仍然允许经典数学发生，通过只考虑我们称为命题的类型。这些类型最多有一个元素，这仅对这些类型恢复证明无关性。我们后面会注意到，我们允许对命题假定排中律。

1.2.5 同伦论

同伦论起源于拓扑学，并与高阶范畴论有联系。它的主要研究对象是同伦的概念。同伦本质上是两个拓扑映射 $f, g : X \to Y$ 之间的变形。经典地，它是一个连续映射 $H : X \times [0,1] \to Y$，满足 $H(-,0) = f$ 和 $H(-,1) = g$。在这种情况下，人们写作 $f \simeq g$ 并说它们是同伦的。这是一个“等价关系”，但是同一对函数的两个同伦 $H_1, H_2 : f \simeq g$ 不一定是相同的。事实上，可以研究同伦之间的同伦 $\alpha : X \times [0,1] \times [0,1] \to Y$，满足 $\alpha(-,-,0) = H_1$ 和 $\alpha(-,-,1) = H_2$。重复这个过程揭示了 ∞-群胚的无限结构。

由无公理基础计划 [1] 引入的同伦类型论非常好地描述了这种情况。它是拓扑学中分析同伦论的综合复制品。该理论的规则可以写在一张 A5 纸上，更高的结构优美地从相等类型的消去规则中推导出来。我们后面会看到，由于这种更高结构，一个只有一个被证明元素的类型可能非常有趣。这个示例类型类似于 S^1，并包含预期的更高结构群 $\mathbb{Z}$。

1.2.6 序理论、范畴论与高阶范畴论

在 R. Harper 等人 [19] 的讲座中，建立了序理论、普通范畴论和类型论之间的有趣联系。许多概念在这三个理论中都能找到，类似于我们之前看到的 Curry-Howard 元理论同构。例如，乘积类型对应于范畴论中的乘积，在序理论中这对应于交运算。这对偶和 Yoneda 引理也是如此。此外，Heyting 代数自然地从基本类型论中产生，是序理论的研究对象之一。它们也自然地作为特殊范畴存在，并出现在证明论中。

这项研究的自然延续是将其从 ∞-群胚扩展到 ∞-范畴。我们现在讨论两种可能的方法来添加方向性并使态射不可逆。

1.3 向 HoTT 添加方向性的两种方法

我们已经提到，HoTT 以特别简单而优美的方式表示了同伦论。对于添加方向性并在类型论中展示综合 ∞-范畴的方法，情况则不同。虽然这两种理论本身很有用，但与 HoTT 和 ∞-群胚的情况相比，它们有所欠缺。产生类似的情况将是该研究领域的“圣杯”，并且很可能不存在。在本节中，我们介绍两种方法，它们各自都有缺陷。第一种方法违反了某些范畴论的预期，第二种方法不建立在 HoTT 之上，而是需要修改，使其本质上不同于 Martin-Löf 类型论。

在本论文中，我们研究第一种方法，并在第 3.8 节详细解释违反了哪个范畴论预期。这种方法的好处是 HoTT 中的所有陈述仍然成立。此外，使用证明辅助工具 rzk [4] 的形式化工作涵盖了 [3] 中相当一部分理论，包括 U. Buchholtz 和 J. Weinberger [5] 以及 C. Bardomiano Martinez [6] 的工作。

1.3.1 用形状扩展 HoTT

我们非常简要地总结 E. Riehl 和 M. Shulman [2] 添加方向性的方法：我们在类型论中添加另一层包含形状的内容，本质上是单纯形 Δ^n 和某些子形状，例如它们的边界。然后我们将 $\hom_A(x,y)$ 定义为从 $\Delta^1 \to A$ 的映射的类型，这些映射在初始顶点处为 x，在终端顶点处为 y。这种构造产生的类型不是范

畴，因为这些态射的先验组合是不可能的。然而，如果限制自己研究 Rezk 类型，该理论允许我们证明关于 $(\infty, 1)$-范畴的几个陈述。

除了类型不是范畴的问题之外，我们无法构造一个合理的范畴范畴。更糟糕的是，笔直化与反笔直化的 Grothendieck 构造在范畴意义上不起作用。我们将在第 3.8 节详细探讨这一点。

扩展 HoTT 的优点在于所有现有陈述仍然成立，这很好，因为 HoTT 已被证明是一个有用的理论。

1.3.2 限制 Π-类型

另一种方法是本质上限制 Π-类型的形成。这些是类型论中对应全称量词的概念，并推广了普通函数。并非所有依赖类型都应该被允许转化为 Π-类型。D.-C. Cisinski, B. Cnossen, K. Nguyen, and T. Walde [20] 以非类型论的方式探索了这一点，但应该可以从公理中提取出一个类型论。这种方法的优点在于它真正展示了综合范畴论。他们的工作甚至没有定义 “类型” 的概念。存在一个范畴的范畴，并且 Grothendieck 构造按预期工作。

这种方法的缺点是只有余笛卡尔纤维化支持幂对象（参见 [20, 公理 N]）。这对应于类型论中 Π-类型的形成，因此 Π-类型仅对作为余笛卡尔族的依赖类型存在。这意味着特别地，并非所有 HoTT 中的陈述都可以转移，并且所有陈述都需要在依赖它们之前仔细检查（例如通过使用证明辅助工具证明它们）。关于这一理论的工作尚未完成，也没有证明辅助工具形式化这一理论，我们在此不再进一步研究它。

1.3.3 圣杯

由于两种方法都放弃了另一世界的一些东西，我们无法重现 HoTT 和 ∞-群胚的成功。范畴友好的限制 Π-类型的方法很可能产生更自然的范畴论环境。然而，形式化方法是否也能在那里表现良好仍有待观察。

1.4 动机

高阶范畴论中一个出现在许多地方的概念是余笛卡尔纤维化。它们在 D.-C. Cisinski, B. Cnossen, K. Nguyen, and T. Walde [20] 中扮演了重要角色，出现在三个公理中。在 E. Riehl 和 M. Shulman [2] 中，它们是第 2.8 节讨论的忠实 Grothendieck 构造的核心障碍的一部分。此外，余笛卡尔纤维化在高阶范畴论中通常具有重大意义，因为它们允许构造无穷范畴之间的函子，正如 M. Land [21] 所讨论的那样。

基于这些原因，我们在 rzk 中形式化了余笛卡尔纤维化，并将我们的工作上游集成到 [3] 中。我们特别遵循了 U. Buchholtz 和 J. Weinberger [5] 以及 J. Weinberger [7] 的工作，他们将余笛卡尔纤维化作为 LARI 族的特例进行研究。

1.5　综合 vs 分析

我们已经提到，我们采用综合的方法做数学。它是分析方法的对偶——综合方法旨在公理化地产生一个行为正确的定义，而不是在集合论内部构造一个理论。一个著名的关于自然数的综合理论是皮亚诺公理。这些公理通过声明“0”存在来定义它；此外，对于任何自然数，都存在一个与其之前所有数不同的后继。从这些少数性质（以及归纳法等更多公理）出发，可以推导出许多关于自然数的有用陈述。相比之下，定义自然数的分析方法例如冯 · 诺依曼构造，它嵌入在 Zermelo-Fraenkel 集合论中。自然数被定义为集合，从 $0 := \emptyset$ 开始，n 的后继为 $n \cup \{n\}$。加法、减法和任何其他运算都使用基本集合运算来定义。

分析方法允许使用现有理论，并表明数学概念存在于该理论中。然而，将它们嵌入另一个理论的需要使得它们本质上更加冗长。通常，人们必须“深入”定义，并使用关于具体表示的事实来得出所需陈述的证明。这在形式化数学时尤其如此。当在一个将实数构造为 Cauchy 序列等价类的理论中将 2 和 1 相加时，证明辅助工具有两种选择。要么它帮助解构序列，逐元素相加，然后观察到结果序列也是常数。要么，数学家必须执行这些额外的工作，而证明辅助工具只检查该工作的正确性。综合方法通过专门为概念构建理论来弥补这个问题，从而得到更清晰的证明和更大的普适性。在任何公理可以被分析证明的理论中，我们可以依赖从综合理论中证明的任何陈述。

1.6　论文结构

第 2 节介绍了同伦类型论，并涵盖了来自一般类型论的许多重要概念。我们遵循无公理基础计划 [1] 和 R. Harper 等人 [19] 的工作，并解释 rzk 如何实现 HoTT。第 3 节在此基础上，添加了 E. Riehl 和 M. Shulman [2] 的扩展类型，并介绍了来自范畴论的许多重要概念。在第 4 节中，我们讨论了正交纤维化和族，以便定义内族。这些是余笛卡尔纤维化所需的基本概念。我们还在本部分研究了理论中正方形的行为。接下来，我们在第 5 节、第 6 节和第 7 节中介绍我们论文的主要工作，其中我们首先介绍了始截面以及依赖始截面的概念。我们证明它们都等价于 LARI 伴随，U. Buchholtz 和 J. Weinberger [5] 使用 LARI 伴随来定义 LARI 族。在涵盖余笛卡尔纤维化之前的最后一步，我们研究了 LARI-胞腔，这是 J. Weinberger [7] 创造的一个概念，概括了余笛卡尔纤维化。我们证明这些等价于我们之前定义的 LARI-族。最后，在第 7.4 节中，我们收集了抽象劳动的成果，并讨论了 i_0-LARI 族作为余笛卡尔纤维化的具体实例化。最后，在第 8 节中，我们解释了如何使用 rzk 形式化我们的大部分工作。在我们的形式化过程中，我们发现了一个 rzk 中的 bug，维护者迅速修复了它。我们在附录 A 中包含了一个词汇表，其中包含用于查找符号记法的快速参考。

第二章　同伦类型论

在本节中，我们遵循 [1] 对同伦类型论（HoTT）进行总结。我们涵盖了在后续章节中将要用到的所有内容。我们还解释了某些概念在 rzk 中是如何实现的。关于具体例子、更多背景和更长的解释，我们请读者参阅 [1] 和 [19]。

本节非常注重形式正确性，我们经常明确提及一些在后面会忽略的细节。我们通过在证明辅助工具 rzk [4] 中证明了我们的陈述，来为后面部分的宽松处理提供理由。

2.1　类型论基础

HoTT 很大程度上基于 Martin-Löf [10] 发展的类型论。尽管从他最初的论文发表已经过去了几十年，并且许多其他数学家也做出了贡献，但最初的思想仍然在我们的理论中可见。对于本小节，我们主要遵循 R. Harper 等人 [19] 的讲座。

在开始写下我们的理论之前，我们需要涵盖几个概念。类型论起源于逻辑学，并与构造性数学和证明相关数学有联系。例如，我们不会假定排中律成立，因为它与我们后面要介绍的幺半等价性公理直接冲突。逻辑学与类型论之间存在一个基本的联系，由 Curry-Howard 同构描述。它将逻辑学和类型论中的概念一一对应起来：

逻辑概念	**类型论概念**
命题	类型
命题 P 的证明	类型 P 的项
真 $\top$	单元类型 $\mathbb{1}$
假 $\bot$	空类型 $\mathbb{0}$
合取 $A \wedge B$	乘积类型 $A \times B$
析取 $A \vee B$	不交并类型 $A + B$
蕴涵 $A \to B$	函数类型 $A \to B$
存在量词 $\exists x.P$	Σ-类型 $\Sigma(x : X, P)$
全称量词 $\forall x.P$	Π-类型 $(x : X) \to P$

表 2.1: 逻辑与类型论之间的 Curry-Howard 元理论同构

该表是我们本节将介绍的各种类型的重要动机来源。它们代表了有用的数学理论应该具备的某些逻辑概念。

在深入定义这些类型的具体细节之前，我们首先解释如何建立形式语言，以便能够写出它们。

2.1.1　推理规则与判断

为了写下数学理论，存在几种不同的符号系统。例如，集合论使用一阶逻辑。由于类型论不建立在任何逻辑之上，我们使用逻辑学家通常用来描述一阶逻辑（以及许多其他类型的逻辑）的相同符号系统来引入其原始概念。

这个符号系统是一种形式语言，在其中我们定义判断和推理规则（通常简称为规则）。判断是“我们可以在理论内证明的陈述”。规则是我们从其他判断推导出判断的方式。一条规则具有以下语法：

$$\frac{J_1 \quad J_2 \quad J_3 \quad \cdots \quad J_n}{J}\ R_1$$

这条规则名为 R_1。J_i 和 J 是判断。横线上方的 J_i 是规则的前提，底部的 J 是结论。我们允许零前提的规则，这些规则本质上对应于我们理论的公理，因为它们没有前提条件。一条规则可用于在其所有前提成立的情况下推导出其结论。我们的判断中有变量，这些变量在规则中本质上是“全称量化的”。这不是理论内部量词，而是允许对这些规则进行句法替换的结果。本质上它们是宏，我们可以为变量代入任何值，并且它们对我们能代入的任何值都成立。

规则用于构建判断的证明。要证明一个判断，我们需要写下一系列步骤，其中只允许应用推理规则。最后我们应该得出所需判断成立的结论。逻辑学家为此发明了特殊的符号：证明树。一个简单的例子可以在示例 2.1.1 中找到。推理规则可以像乐高积木一样组合，将一条规则的结论插入另一条规则的前提。如

果最终树的所有叶子都是横线（表示它们在没有前提的情况下证明了结论），我们就可以得出底部的判断可以从公理推导出来的结论。

例 2.1.1. 基本的证明树

$$\dfrac{\dfrac{\dfrac{}{J_1} \quad \dfrac{}{J_2}}{J_3} \quad \dfrac{}{J_4} \quad \dfrac{\dfrac{}{J_1} \quad \dfrac{\dfrac{}{J_2} \quad \dfrac{}{J_4}}{J_6}}{J_5}}{J_7}$$

对此的一个趣味性解读是像国际象棋这样的游戏。我们的公理是棋子的起始位置，规则是规定棋子如何移动的游戏规则。一个判断就是棋子的某个特定位置。一个证明是一系列规则，按顺序应用于起始位置后，得到我们想要证明的位置。

一个理论的所有规则都需要被证明为合理的，因为我们声明它们为真而不加证明。严格来说，我们还需要证明系统本身。关于这一点可以给出的论证超出了数学的范畴。我们将其留给哲学家，并满足于这样一个简单的事实：这个系统允许我们描述非常有趣和有用的数学概念。

玩具逻辑

在深入探讨我们类型论的具体规则和判断之前，我们先快速给出一个（不完整的）命题逻辑的例子。这是一个简单的判断和推理规则系统，将使读者有机会在我们本节后面更复杂的类型论规则之前，获得对逻辑语言更直观的理解。我们还将引入非常重要的蕴涵、上下文等概念。

我们首先需要描述玩具逻辑的判断是什么。我们拥有的第一种判断只是普通变量 $A, B, C, \ldots$；我们也允许带下标的变量。

下一种判断也会在我们后面的类型论中出现（尽管形式更复杂）。它被称为蕴涵，并且要求我们定义一个新的句法概念：上下文。一个上下文只是一个变量列表。我们使用大写希腊字母 (Γ, Ξ, Θ) 表示上下文。蕴涵则允许我们表示一个判断可以从一个变量列表推导出来。我们写作 $A_1, \ldots, A_n \vdash B$，其中 A_i 是蕴涵的前提，B 是结论。当我们写 $\Gamma \vdash B$ 时，读作“在上下文 Γ 中，B 成立”。蕴涵主要由以下两条规则支配：

$$\dfrac{}{A \vdash A}\mathrm{R} \qquad \dfrac{\Gamma \vdash A \quad \Gamma, A \vdash B}{\Gamma \vdash B}\mathrm{T}$$

第一条规则是自反性：我们逻辑中的所有陈述必须能从自身推出。第二条规则是传递性，它对应于将引理的证明代入假设该引理为真的证明中。除了这两条规则，还有三条规则允许操作蕴涵的上下文：

$$\dfrac{\Xi \vdash B}{\Gamma, \Xi, \Theta \vdash B}\mathrm{W} \qquad \dfrac{\Gamma, A, \Xi, A, \Theta \vdash B}{\Gamma, A, \Xi, \Theta \vdash B}\mathrm{C} \qquad \dfrac{\Gamma \vdash A}{\pi(\Gamma) \vdash A}\mathrm{P}$$

第一条规则称为弱化，允许向上下文添加不必要的变量。第二条是收缩，允许丢弃重复的变量。最后一条规则是置换，允许重新排序上下文；其中 $\pi(\Gamma)$ 表示“将置换 π 应用于 Γ 的元素”。

我们以易于应用的方式表述了这些规则。使用以下不同的规则就足够了，但那样我们会有更多的中间步骤：

$$\frac{\Gamma \vdash B}{\Gamma, A \vdash B}\text{W'} \qquad \frac{\Gamma, A, A \vdash B}{\Gamma, A \vdash B}\text{C'} \qquad \frac{\Gamma, A, B, \Xi \vdash C}{\Gamma, B, A, \Xi \vdash C}\text{P'}$$

这就完成了我们玩具逻辑中的所有判断，我们可以进入命题逻辑的第一个原始概念：合取。我们引入表达式的新句法。变量是表达式，给定两个表达式 E_1, E_2，我们声明 $E_1 \wedge E_2$ 也是一个表达式。为了赋予这个句法其语义含义（匹配合取的直觉），我们需要三条规则：

$$\frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \wedge B} \wedge\text{-I} \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash A} \wedge\text{-E}_1 \qquad \frac{\Gamma \vdash A \wedge B}{\Gamma \vdash B} \wedge\text{-E}_2$$

第一条规则称为引入规则，它允许我们在合取的两个部分都为真时构造一个合取。第二和第三条规则是消除规则，它们允许我们从合取中证明两个组成部分。

由于在所有这些规则中，上下文 Γ 作为顶部和底部的唯一上下文出现，它不传递任何新信息。根据这个论点，我们可以省略它。因此，如果可能，我们也允许以局部规则形式书写规则：

$$\frac{A \quad B}{A \wedge B} \wedge\text{-I} \qquad \frac{A \wedge B}{A} \wedge\text{-E}_1 \qquad \frac{A \wedge B}{B} \wedge\text{-E}_2$$

除了合取，我们的玩具逻辑中还有蕴含。给定两个表达式 E_1, E_2，我们声明 $E_1 \to E_2$ 也是一个表达式。引入和消除规则是：

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \to B} \to\text{-I} \qquad \frac{A \to B \quad A}{B} \to\text{-E}$$

我们注意到引入规则必须用显式的上下文书写，因为我们使用了两个不同的上下文。消除规则通常称为“肯定前件”。

最后，我们允许带括号的表达式，所以如果 E 是一个表达式，那么 (E) 也是一个表达式。我们禁止任何导致运算顺序歧义的表达式（也可以定义运算之间的优先级以避免不必要的括号）。

使用这些规则，我们现在可以从空上下文证明陈述 $A \to (B \to (A \wedge B))$：

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{A \vdash A}\text{R}}{A, B \vdash A}\text{W} \quad \dfrac{\dfrac{}{B \vdash B}\text{R}}{A, B \vdash B}\text{W}}{A, B \vdash A \wedge B} \wedge\text{-I}}{A \vdash B \to (A \wedge B)} \to\text{-I}}{\vdash A \to (B \to (A \wedge B))} \to\text{-I}$$

除了这些概念，我们还需要允许书写定义。我们为此操作使用冒号和等号“=”。它为右边的内容引入了一个左边的缩写。例如：

$$X := A \to B$$

这意味着以后任何时候看到 X，都应该用 $A \to B$ 替换它。替换时需要注意运算符优先级，我们在玩具逻辑中没有定义。

后面会再次出现的两个逻辑概念是析取、假和真。这两个常量有特别简单的规则：

$$\frac{}{\top}\ \top\text{-I} \qquad\qquad \frac{\bot}{A}\ \bot\text{-E}$$

真引入说在任何上下文中我们都可以推导出真。它没有消除规则，我们不能从“真”证明任何东西。对偶地，假显然没有引入规则。它的消除规则假定假，并允许证明任何原始判断 A。

析取对读者来说应该很熟悉。引入规则也非常简单，反映了与合取的对偶性。一个潜在的新颖之处是消除规则。

$$\frac{A}{A \vee B}\ \vee\text{-I}_1 \qquad \frac{B}{A \vee B}\ \vee\text{-I}_2 \qquad \frac{\Gamma, A \vdash C \qquad \Gamma, B \vdash C}{\Gamma, A \vee B \vdash C}\ \vee\text{-E}$$

消除规则允许我们在给定 $A \vee B$ 的情况下，当我们能同时从 A 和 B 证明 C 时，证明任何陈述 C。这里一个有趣的点是，这给我们的理论增加了一些模糊性，因为我们在调用这条规则时必须选择 C。当我们后面讨论乘积类型时，由于我们类型论中的证明项，这种模糊性会消失。在我们转向类型论蕴涵之前，我们给出一个关于类型论中推理规则含义的非常重要的评论。

2.1.2　规则与类型检查

类型论中的推理规则扮演的角色比逻辑学中重要得多，因为它们不直接用于构造证明。类型论中的规则决定哪些项是有效的；这称为类型检查。然而，这些规则连同项的句法被设置成可以自动应用的方式。

由于这个性质，类型论自然适合借助计算机进行形式化。类型检查可以通过一个程序实现，它让用户提供有效的项。

在本论文的其余部分，我们将不再显式地使用这些规则。相反，我们声称某个项是有效的，并且由于检查这一事实只是一个算法练习，我们在本节之后不再给出证明树。

这是类型论的优点，也是相对于逻辑学的一大进步。证明树是一个笨拙的对象——项则轻便得多，也更容易写下来（想出正确的项仍然很困难，就像想出一个证明很困难一样）。

这一点得到了进一步证实：在我们用来形式化结果的证明辅助工具 rzk [4] 中，没有办法显式地使用推理规则。它们都由类型检查器完全实现，对用户不可见。只有当用户形成了一个无法通过类型检查的项，需要报告错误时，类型检查才显示给用户。rzk 会打印一个步骤的堆栈跟踪，解释预期的情况。我们在第 8 节更详细地解释这一点。

2.1.3　类型论蕴涵

我们首先从一些句法开始定义类型论，然后引入上下文。它们比在命题逻辑中扮演着更重要的角色，因为它们包含更多信息：它们允许我们定义依赖类型和项，这在我们理论中起着至关重要的作用。

我们写 Γ ctx 来声明一个上下文。我们用 $\cdot$ 表示空上下文，当它的缺失会引起混淆时使用。为了描述什么是上下文，我们还需要类型声明的句法构造。我们写 $\Gamma \vdash A$ **type** 来表示 A 在上下文 Γ 中是一个类型。此外，我们用 $a : A$ 表示项声明，说明项 a 具有类型 A。上下文的创建则由以下规则支配：

$$\frac{}{\cdot\ \mathbf{ctx}}\ \mathbf{ctx}\text{-I}_1 \qquad\qquad \frac{\Gamma\ \mathbf{ctx} \qquad \Gamma \vdash A\ \mathbf{type}}{\Gamma, a : A\ \mathbf{ctx}}\ \mathbf{ctx}\text{-I}_2$$

评论 2.1.2. 我们注意到，将形成上下文的规则指定为推理规则在技术上使我们的类型论成为一个多类理论。因为当我们后面写下一条推理规则时，那里的上下文形成需要遵循我们上面写的规则。这并不罕见，事实上 [19] 在没有提及的情况下就这样做了。

当我们为某个判断 J 写 $a : A \vdash J$ 时，这意味着变量 a 被允许出现在该判断中。当我们后面添加宇宙的概念时，在使用 J 时可以摆脱这种隐式依赖。

接下来我们给出蕴涵的自反性和传递性规则。类型论中的蕴涵传递性规则采用了略有不同的形式，需要一个新的句法概念，称为替换。它也更常被称为替换规则：

$$\frac{\Gamma \vdash A\ \mathbf{type}}{\Gamma, x : A, \Xi \vdash x : A}\ \text{R} \qquad\qquad \frac{\Gamma \vdash A\ \mathbf{type} \qquad \Gamma, x : A, \Xi \vdash \mathbf{J} \qquad \Gamma \vdash a : A}{\Gamma, \Xi[x \mapsto a] \vdash \mathbf{J}[x \mapsto a]}\ \text{S}$$

句法 $[x \mapsto a]$ 将替换应用于直接写在它前面的变量。替换将 x 的所有出现改为 a。我们允许写 $M[x \mapsto a, y \mapsto b]$ 代替 $(M[x \mapsto a])[y \mapsto b]$。

评论 2.1.3. 替换由于需要区分自由变量和绑定变量而使理论复杂化。我们完全忽略这个技术细节，因为有一个规范的解决方案：不替换绑定变量。这可以由计算机实现。或者，我们可以只使用每个变量一次。为了方便，我们允许重复使用变量。要找到变量的类型，在向上遍历子表达式树时，只考虑最近的声明。

如果一个变量不需要为了项通过类型检查而将其添加到上下文中，那么该变量是绑定的。自由变量是需要存在于上下文中才能使项通过类型检查的变量。

我们没有单一的弱化规则，而是给出一个规则模式。对于每个适合该句法位置的判断 J，我们都有一个采用以下形式的推理规则：

$$\frac{\Gamma, \Xi \vdash \mathbf{J} \qquad \Gamma \vdash A\ \mathbf{type}}{\Gamma, x : A, \Xi \vdash \mathbf{J}}\ \text{W}$$

收缩和置换也必须是规则模式：

$$\frac{\Gamma \vdash A\ \mathbf{type} \qquad \Gamma, x : A, y : A, \Xi \vdash \mathbf{J}}{\Gamma, z : A, \Xi[x \mapsto z, y \mapsto z] \vdash \mathbf{J}[x \mapsto z, y \mapsto z]}\ \text{C} \qquad\qquad \frac{\Gamma \vdash A\ \mathbf{type} \qquad \Gamma \vdash B\ \mathbf{type} \qquad \Gamma, a : A, b : B, \Xi \vdash \mathbf{J}}{\Gamma, b : B, a : A, \Xi \vdash \mathbf{J}}\ \text{P}$$

置换规则仅在 a 不出现于 B 且 b 不出现于 A 时才能应用。

评论 2.1.4. 与我们的玩具逻辑相比的新颖之处在于我们可以有依赖项/类型。例如，如果我们有一个自然数类型 $\mathbb{N}$，我们可以声明长度为 n 的序列的依赖类型，其中 n 是一个自然数。用我们的形式语言，我们可以通过判断来表达：

$$A\ \mathbf{type}, n : \mathbb{N} \vdash \mathrm{Seq}_n(A)\ \mathbf{type}$$

在 rzk 中，没有蕴涵。它使用 Π-类型和宇宙（我们稍后将介绍两者）来代替依赖类型。定义也允许指定一个上下文，该上下文会被翻译成 Π-类型。

评论 2.1.5. 关于类型论的证明论观点是，类型的元素是类型所代表的陈述的证明。我们可以有同一陈述的多个不相等的证明，这使得我们的理论是证明相关的。注意，经典数学是证明无关的，从理论内部不可能区分同一陈述的两个不同证明。甚至一开始就不可能比较它们，因为一个判断要么成立，要么不成立。

一个拥有元素的类型被称为是居留的，它对应于经典数学中的一个真陈述。没有任何证明的类型被称为非居留的，它对应于假。

我们玩具逻辑缺少的另一个概念是相等性，所以我们接下来介绍它。

2.1.4 判断相等

我们要介绍的下一个概念是判断相等。它也被称为定义相等。它描述了两个句法元素何时是“相同的”。这意味着在我们的整个理论中，我们应该能够在任何地方用一个替换另一个而不引起语义变化。我们需要将这种相等性与命题相等区分开来，命题相等我们稍后将使用普通的 $=$ 符号作为一种类型引入，而判断相等使用 $\equiv$。$=$-类型是同伦类型论中最重要的类型，它赋予了理论有趣的无限结构。

我们写 $\Gamma \equiv \Gamma'$ 表示两个上下文 Γ 和 Γ' 判断相等。对于类型，如果 $\Gamma \vdash A\ \mathbf{type}$ 且 $\Gamma \vdash A'\ \mathbf{type}$，我们写 $\Gamma \vdash A \equiv A'$ 表示 A 和 A' 在上下文 Γ 中判断相等。最后，我们也允许对类型的元素进行相等比较，所以给定 $\Gamma \vdash a : A$ 和 $\Gamma \vdash a' : A$，我们写 $\Gamma \vdash a \equiv a'$ 表示 a 判断等于 a'。因此我们还需要 $\equiv$ 的三条自反性规则。

$$\frac{\Gamma\ \mathbf{ctx}}{\Gamma \equiv \Gamma}\ \equiv\text{-R}_1 \qquad \frac{\Gamma \vdash A\ \mathbf{type}}{\Gamma \vdash A \equiv A}\ \equiv\text{-R}_2 \qquad \frac{\Gamma \vdash A\ \mathbf{type} \qquad \Gamma \vdash a : A}{\Gamma \vdash a \equiv a}\ \equiv\text{-R}_3$$

判断相等是一个非常基本的概念，因此它需要与我们添加到形式语言中的其他每个概念具有兼容性规则。对于类型（我们尚未介绍），这些规则称为 η 和 β 规则，我们在定义每种类型时给出它们。为了使判

断相等与蕴涵兼容，我们需要以下规则（我们不给出名称，因为我们不显式使用它们）：

$$\frac{\Gamma \vdash \boldsymbol{J} \qquad \Gamma \equiv \Gamma'}{\Gamma' \vdash \boldsymbol{J}} \qquad\qquad \frac{\Gamma \equiv \Gamma' \qquad \Gamma \vdash A \equiv A'}{\Gamma, x : A \equiv \Gamma', x : A'}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash a' : A \qquad \Gamma \vdash a \equiv a' \qquad \Gamma, x : A \vdash B \ \mathbf{type}}{\Gamma \vdash B[x \mapsto a] \equiv B[x \mapsto a']}$$

$$\frac{\Gamma \vdash a : A \qquad \Gamma \vdash a' : A \qquad \Gamma \vdash a \equiv a' \qquad \Gamma, x : A \vdash b : B}{\Gamma \vdash b[x \mapsto a] \equiv b[x \mapsto a']}$$

第一条规则是一个模式，允许我们从相等的上下文中证明相同的事情。第二条规则描述了我们如何证明上下文的相等性。第三和第四条规则表明，在替换的值相等的情况下，依赖类型和项是相等的。

在 rzk 中，判断相等也使用 $\equiv$ 符号。它的类型检查器中有相同的替换规则，因此允许在所有情况下交换判断相等的值。

2.1.5　一般句法

我们使用 $:=$ 进行定义，写 $X := a$ 引入 X 作为 a 的简写符号（这里意义不大，但 a 可以是任何表达式，而 X 需要是一个标识符）。

在 rzk 中使用了非常相似的符号：

```
#def X : A := a
```

这里除了 X 的表示之外，还需要指定它的类型。在指定类型的 `: A` 前面，我们允许放置上下文变量。句法要求将它们放在括号中：

```
#def X
    ( a : A)
    : A
    := a
```

在我们的理论中，我们也有带括号的表达式，所以如果 E 是一个表达式，(E) 也是一个有效的表达式。在某些情况下，我们允许省略严格来说语法上需要的括号。例如，将函数应用于元组时，我们允许写 $f(a, b)$ 而不是 $f((a, b))$。这与将带有两个参数的函数应用于两个值有歧义，但无论如何这些函数类型通过柯里化是等价的，所以我们不会遇到问题。

2.1.6　单位类型

我们介绍的第一个类型将是非常简单的。它被称为单位类型，写作 $\mathbb{1}$，它恰好由一个元素组成，我们使用语法 $*_{\mathbb{1}}$。

定义 2.1.6 (单位类型)**.**

$$\frac{}{\mathbb{1}\ \mathbf{type}}\ \mathbb{1}\text{-F} \qquad\qquad \frac{}{*:\mathbb{1}}\ \mathbb{1}\text{-I}$$

由于我们也会将 $*$ 的表示法用于其他只有一个成员的类型，如果有歧义或者我们想更明确地说明我们讨论的是哪个 $*$，我们会写 $*_{\mathbb{1}}$。

这个类型只有一个 η 规则，没有 β 规则：

$$\frac{x:\mathbb{1}}{x\equiv *}\ \mathbb{1}\text{-}\eta$$

这表明单位类型的任何值都必须判断等于 $*$。

这个类型不是特别有趣，因为它是最平凡的类型。它表示逻辑中 $\top$/真的值。从范畴论来看，它是 Cat 中的终端对象。只有一个函子映射到 $\mathbb{1}$（注意这个唯一性取决于判断相等!）。

在 rzk 中，它名为 Unit，该类型的唯一值称为 unit。它也有 η 规则，所以每当我们有一个依赖于 $x:\mathsf{Unit}$ 的定义时，我们知道 $x\equiv\mathsf{unit}$。

2.1.7　空类型

空类型表示假，写作 $\mathbb{0}$，没有元素。在它存在的情况下，我们可以证明任何类型的居留性。

定义 2.1.7 (空类型)**.**

$$\frac{}{\mathbb{0}\ \mathbf{type}}\ \mathbb{0}\text{-F} \qquad\qquad \frac{A\ \mathbf{type} \qquad x:\mathbb{0}}{x:A}\ \mathbb{0}\text{-E}$$

没有 β 或 η 规则。

我们注意到这个类型在 rzk 中不可用。从范畴论来看，它是 Cat 中的始对象，由于消去规则，我们总是可以像预期那样定义到任何任意类型的函数。

2.1.8　Σ 类型

我们介绍的 Martin-Löf 类型论中的下一个类型是 Σ-类型。它是对乘积类型的推广，我们首先介绍乘积类型。形成乘积类型的句法是 $A\times B$。我们还有两个投影函数，称为 π_1 和 π_2。乘积类型的规则（以局部规则形式给出）是：

$$\frac{A\ \mathbf{type} \qquad B\ \mathbf{type}}{A\times B\ \mathbf{type}}\ \times\text{-F} \qquad \frac{a:A \qquad b:B}{(a,b):A\times B}\ \times\text{-I} \qquad \frac{p:A\times B}{\pi_1(p):A}\ \times\text{-E}_1 \qquad \frac{p:A\times B}{\pi_2(p):B}\ \times\text{-E}_2$$

这些规则从左到右依次称为乘积类型的形成规则、引入规则和消除规则。大多数类型都有所有这三种规则。形成规则描述了我们什么时候甚至可以谈论乘积类型。我们从规则中省略了所有非依赖的“类型”判断，因为它们非常分散注意力且不包含新信息（总是可以从上下文中推断出来）。第二条规则是引入规则。它说明我们如何引入乘积的值。接下来的两条规则是其对应物，消除规则。它们规定了可以用乘积类型的值做什么。注意我们如何使用其局部规则形式指定所有这些规则，因为任何地方都没有使用上下文。

注意，虽然 π_1 和 π_2 看起来像函数，但严格来说它们是只是行为类似于函数的句法概念（我们尚未定义函数）。这种区别在实践中无关紧要，在本论文中我们就把它们当作函数对待。

用于与 $\equiv$ 兼容的 β 和 η 规则是：

$$\frac{a : A \qquad b : B}{\pi_1((a,b)) \equiv a}\ \times\text{-}\beta_1 \qquad \frac{a : A \qquad b : B}{\pi_2((a,b)) \equiv b}\ \times\text{-}\beta_2 \qquad \frac{x : A \times B}{(\pi_1(x), \pi_2(x)) \equiv x}\ \times\text{-}\eta$$

另一种写这些规则的方式是通过等式，因为规则的前提从结论中显而易见（本质上我们只需要对所有变量进行全称量化，然后运行类型推断算法）：

$$\pi_1((a,b)) \equiv a \qquad \pi_2((a,b)) \equiv b \qquad (\pi_1(x), \pi_2(x)) \equiv x$$

乘积类型是类型论中对应合取的概念。这是因为当证明 $A \times B$ 的居留性时，需要给出 A 的证明和 B 的证明。

Σ-类型提供的推广是依赖性。第二个分量的类型可以依赖于第一个分量的值。例如，使用 Σ-类型，我们可以形成一个单一的类型，其中包含任意长度的序列。

定义 2.1.8 (Σ-类型)**.** Σ-类型的句法是 $\Sigma(x : A, B)$，其中 A 是一个类型，B 是在变量 $x : A$ 上的依赖类型。这个构造绑定变量 x，该变量允许在 B 中出现。以后当我们有宇宙时，这种依赖性会更清晰地标记为 $\Sigma(x : A, B(x))$，其中 B 是从 A 到宇宙的函数，写作 $B : A \to \mathcal{U}$。

我们有下面的形成、引入和消除规则：

$$\frac{\Gamma \vdash A\ \mathbf{type} \qquad \Gamma, x : A \vdash B\ \mathbf{type}}{\Gamma \vdash \Sigma(x : A, B)\ \mathbf{type}}\ \Sigma\text{-F} \qquad \frac{\Gamma, x : A \vdash B\ \mathbf{type} \qquad \Gamma \vdash a : A \qquad \Gamma \vdash b : B[x \mapsto a]}{\Gamma \vdash (a,b) : \Sigma(x : A, B)}\ \Sigma\text{-I}$$

$$\frac{\Gamma, x : A \vdash B\ \mathbf{type} \qquad \Gamma \vdash p : \Sigma(x : A, B)}{\Gamma \vdash \pi_1(p) : A}\ \Sigma\text{-E}_1 \qquad \frac{\Gamma, x : A \vdash B\ \mathbf{type} \qquad \Gamma \vdash p : \Sigma(x : A, B)}{\Gamma \vdash \pi_2(p) : B[x \mapsto \pi_1(p)]}\ \Sigma\text{-E}_2$$

此外，β 和 η 规则与乘积类型相同（但现在如果我们进行类型检查，有 $b : B[x \mapsto a]$ 而不是 $b : B$）：

$$\pi_1((a,b)) \equiv a \qquad \pi_2((a,b)) \equiv b \qquad (\pi_1(x), \pi_2(x)) \equiv x$$

我们允许写 $\Sigma(a : A, b : B, C)$，它应解释为 $\Sigma(a : A, \Sigma(b : B, C))$。对于项我们也做同样处理，所以 $(a,b,c) \equiv (a,(b,c))$。在内部类型占用大量垂直空间的情况下，我们也使用更大的“$\sum$”符号。

类型论文献中经常使用略有不同的 Σ-类型符号：

$$\sum_{x:A} B \equiv \Sigma(x:A,B)$$

我们选择自己的符号，因为它避免了索引漂移，例如比较下面两种表示法的类型：

$$\sum_{x:\sum_{y:\sum_{a:A} B} C} D \quad \text{vs} \quad \Sigma(x:\Sigma(y:\Sigma(a:A,B),C),D)$$

例 2.1.9. 我们给出一个 Σ-类型的基本例子。假设我们有自然数类型 $\mathbb{N}$ 和长度为 $n:\mathbb{N}$ 的序列类型 $\mathrm{Seq}_n(A)$，其中 A 是一个类型。

我们现在可以形成 Σ-类型：

$$\Sigma(n:\mathbb{N},\mathrm{Seq}_n(A))$$

这个类型是任意长度序列的类型。它的一个元素由长度 $n:\mathbb{N}$ 和一个该长度的序列组成。

这个类型在逻辑上等价于陈述“存在一个自然数，使得存在一个 A^n 中该长度的序列”。这个陈述非常平凡，表明该类型的所有有趣性质必须来自元素之间的差异。

我们的符号对和类型的两个分量赋予相同的优先级，而求和符号使得第二个分量重要得多。此外，我们的符号更接近 rzk 中使用的符号：

```
Σ (a : A) , B
```

这种符号的一个缺点是，在嵌套层次较高时，很难辨别哪个括号属于哪个 Σ。我们使用更大的括号、向量表示法或特殊换行/格式来防止影响可读性。在编写 rzk 代码时，[3] 使用的特殊格式确保任何嵌套层次都易于识别。

正如我们之前提到的，Σ-类型对应于逻辑中的存在量词：为了给出 $\Sigma(x:A,B)$ 的一个元素（从而证明 $\exists x:A,B$ 成立），首先必须提供一个见证 $a:A$，然后是一个证明 $b:B[x\mapsto a]$。这个证明 b 表明对于我们的见证 a，类型 B（其中 x 被替换为 a）是居留的（逻辑上说：见证 a 满足条件 B）。

为了恢复普通的独立乘积类型，我们只需要为 B 选择一个（在 x 上）非依赖的类型。在这种情况下，我们允许写 $A\times B\equiv\Sigma(x:A,B)$（其中 x 不在 B 中自由出现）。

Σ-类型的范畴论版本称为“依赖和”[22]。它不太常见，因为它直接使用我们后面称为类型族的纤维化版本来工作。这使得它使用时更加冗长。它推广了余积的概念。在我们的类型论中，我们可以使用 Σ 类型形成任何范畴极限。

评论 2.1.10. 有时会遇到非常大的 Σ-类型。在这种情况下，将元素以及类型本身写作向量可以显著提高可读性，特别是当每个分量的项包含括号和许多其他子表达式时。向量总是从上到下列出项目。例如：

$$\begin{pmatrix} a \\ b \\ c \\ d \\ e \end{pmatrix} : \Sigma \begin{pmatrix} a:A \\ b:B \\ c:C \\ d:D \\ E \end{pmatrix}$$

我们另外还允许在 Σ-类型中省略元素的名称，当它们不在后面的类型中出现时。这也可以使用 × 来写，但在某些情况下更节省空间。

2.1.9　Π 类型

接下来我们定义 Π-类型。这是函数类型的推广，也可以解释为乘积类型，因此得名。它们在逻辑上的对应物是全称量词 $\forall x.\, P$；如果有一个类型 A 的值 x，那么 $\forall x.\, P$ 的证明“允许我们获得 $P[x \mapsto a]$ 的证明”。符号与 Σ-类型非常相似，但使用大写的 π：$\Pi(a : A, B)$。该类型的值使用经典的 lambda 表示法 $\lambda a : A.\, b$，其中 a 是类型 A 的变量，点在之后绑定，b 是一个项。

乘积解释是：$\Pi(i : I, E_i)$ 的一个值由每个 $i : I$ 的 $e_i : E_i$ 组成。对我们来说，Π-类型作为依赖函数类型的类型论解释是最有用的。这是因为它们表现出的行为类似于连续映射——只允许它们的值以一致的方式变化。我们将在本节后面定义可缩性概念时，以及在第五节的更后面，再次观察到这种有些模糊的行为。

定义 2.1.11 (Π-类型)**.**

$$\frac{A\ \mathbf{type} \qquad \Gamma, x : A \vdash B\ \mathbf{type}}{\Gamma \vdash (x : A) \to B\ \mathbf{type}}\ \Pi\text{-F} \qquad \frac{\Gamma, x : A \vdash B\ \mathbf{type} \qquad \Gamma, x : A \vdash b : B}{\Gamma \vdash \lambda x.\, b : (x : A) \to B}\ \Pi\text{-I}$$

$$\frac{\Gamma, x : A \vdash B\ \mathbf{type} \qquad \Gamma \vdash f : (x : A) \to B \qquad \Gamma \vdash a : A}{\Gamma \vdash f(a) : B[x \mapsto a]}\ \Pi\text{-E}$$

β 和 η 规则是：

$$(\lambda x.\, b)(a) \equiv b[x \mapsto a] \qquad (\lambda x.\, b(x)) \equiv b$$

η 规则（第二个）仅在 x 不在 b 中自由出现时成立。

我们允许写 $(a : A, b : B, c : C) \to D$ 代替 $(a : A) \to (b : B) \to (c : C) \to D$。在独立情况下，我们也写 $A \to B \to C \to D$，这不应与从 A 到 D 的复合函数的通常数学含义混淆。当我们要写复合时，我们总是在箭头上方放置名称。这种 Π-类型的符号再次类似于 rzk 中的符号。传统上，人们会为此类型使用乘积符号：

$$\prod_{a:A} B$$

然而，这也有索引漂移的问题，因此我们更喜欢我们的符号。

对于 lambda 表达式，我们也允许在变量后放置类型以帮助读者理解独立 lambda 表达式的类型：$\lambda x : A.\, b$。我们也允许对 Σ-类型的参数进行模式匹配。假设我们想写一个域为 $\Sigma(a : A, B)$ 的类型，我们这样写：$\lambda(a, b).\, x$，其中 x 可以是任何项。

与 Σ-类型一样，在写类型和定义映射时允许类似向量的表示法：

$$(a : A, b : B, c : C) \to D \equiv \begin{pmatrix} a : A \\ b : B \\ c : C \end{pmatrix} \to D$$

$$(a, b, c) \mapsto d \equiv \begin{pmatrix} a \\ b \\ c \end{pmatrix} \mapsto d$$

类似于 Σ-类型，我们可以恢复标准非依赖函数类型的概念（我们直接写作 $A \to B$）。

范畴论上，这是依赖积 [23]，类似于依赖和，很少被教授，并且是用纤维化而非族来表述的。

定义 2.1.12 (恒等函数)**.** 对于任何类型 A，我们写作 id_A 表示恒等函数 $A \to A$，定义为 $\lambda a.\, a$。

在 rzk 中，它是这样定义的：

```
#def identity
    ( A : U )
    : A → A
    := \ a → a
```

rzk 中 Π 类型的句法是 `(a : A) → B`，对于 lambda 表达式，它使用反斜杠后跟参数，然后是一个箭头，再是 lambda 的主体：`\ x y → t`。

`A : U` 的部分声明 `A` 属于类型 `U`，即类型的宇宙。接下来我们将引入它作为“A **type**”判断的替代。在下一节之后，我们将能够定义一个单一的恒等函数，而不需要为每个类型都给出模式定义。

定义 2.1.13 (函数复合)**.** 给定两个可复合的函数 $f : A \to B, g : B \to C$，我们定义：

$$g \circ f := \lambda a.\, g(f(a))$$

2.1.10　宇宙

宇宙是类型的类型——一个其元素本身就是类型的类型。有几个理由说明为什么人们希望在自己的理论中拥有宇宙。其中之一是 Grothendieck 构造，我们将在第 2.8 节介绍；没有类型宇宙就无法进行该构造。我们也希望拥有宇宙以充分利用依赖类型：宇宙将类型转变为类型的元素，使它们成为理论中的非特殊值。最后，宇宙允许我们减少某些类型所需的推理规则数量。

例 2.1.14. 考虑具有以下规则的布尔类型：

$$\overline{\mathsf{bool}\ \mathbf{type}} \qquad \overline{t : \mathsf{bool}} \qquad \overline{f : \mathsf{bool}}$$

根据我们目前的规则，无法定义一个依赖于布尔值的类型。因此，C 中的变量 b 基本上是无用的，从而 x 和 y 具有相同的类型。然而，如果我们引入宇宙，允许它们具有完全不同的类型是有用的，并且是非常可表达的。

在这种情况下，宇宙的一种替代方案是添加类型层面的 if 表达式：

$$\frac{\Gamma \vdash X\ \mathbf{type} \quad \Gamma \vdash Y\ \mathbf{type} \quad \Gamma \vdash v : \mathsf{bool}}{\Gamma \vdash \mathrm{if}(v, X, Y)\ \mathbf{type}} \qquad \mathrm{if}(t, X, Y) \equiv X \qquad \mathrm{if}(f, X, Y) \equiv Y$$

然后我们可以使用它来定义依赖于布尔值的类型。给定 $x : X$ 和 $y : Y$，我们有 $\mathrm{if}(v, x, y) : \mathrm{if}(v, X, Y)$。这种方法称为“大消去”[19]。这种方法很笨拙，因为现在我们必须为所有这些类型添加两个构造子，而使用宇宙，我们可以简洁地表述。

在我们的理论中，我们用 $\mathcal{U}$ 表示宇宙。为了将其正式引入理论，我们希望添加以下规则：

$$\frac{}{\mathcal{U}\ \mathbf{type}} \qquad \frac{\Gamma \vdash A\ \mathbf{type}}{\Gamma \vdash A : \mathcal{U}}$$

然而，这使得判断 $\mathcal{U} : \mathcal{U}$ 可推导，这将我们引向与集合论和集合的集合相同的困境：罗素悖论（参见 [1, 19]）。为了避免悖论，我们使用宇宙的累积层次。$\mathcal{U}_i$ 是“规模 i”的类型宇宙。更高一层，$\mathcal{U}_{i+1}$ 包含来自前一层的所有类型以及 $\mathcal{U}_i$ 本身。因此，代替上面两个不一致的规则，我们有以下规则：

$$\frac{}{\mathcal{U}_i : \mathcal{U}_{i+1}} \qquad \frac{A : \mathcal{U}_i}{A : \mathcal{U}_{i+1}}$$

然后我们需要用 $A : \mathcal{U}_i$ 替换现有规则中使用 A **type** 原始概念的地方（然后我们可以从理论中完全移除类型原始概念）。例如，现在 Σ-类型的形成规则为：

$$\frac{\Gamma \vdash A : \mathcal{U}_i \quad \Gamma, x : A \vdash B : \mathcal{U}_i}{\Gamma \vdash \Sigma(x : A, B) : \mathcal{U}_i}$$

类似于当上下文不言自明时可以省略它，我们也可以对宇宙的层次做同样处理。事实上，可以实现一个推理算法，自动为所有涉及的宇宙选择尽可能小的层次数字。因此，我们现在通常不带下标地写作 $\mathcal{U}$。如果我们写 $\mathcal{U} : \mathcal{U}$，这意味着左边宇宙的层次必须低于右边的层次。

这些层次是元理论的一部分，理论内部无法访问：在理论内部无法使层次依赖于理论中的值。例如，类型 $(n : \mathbb{N}) \to \mathcal{U}_n$ 不存在，将除层次之外的任何东西放入 $\mathcal{U}$ 的下标中是语法错误。这很重要，因为如果可能，那么我们将必须拥有 $\mathcal{U}_{\mathbb{N}_0}$ 以及更高的序数。

任何使用 $\mathcal{U}$ 的定义或定理都是一个模式定义/定理，因为它对于所有出现的层次下标是元通用的。在实践中这永远不会成为问题，因为我们可以将 $\mathcal{U}$ 视为一个单一类型。

我们现在可以使用宇宙将依赖类型本身展示为一种类型：

定义 2.1.15 (类型族)**.** 给定类型 $A : \mathcal{U}$ 上的一个类型族就是一个函数 $B : A \to \mathcal{U}$。

与依赖类型相比，类型族更方便，因为我们可以将所有来自变量替换的复杂性委托给 λ 项，并且只需为对应于 a 的类型写 $B(a)$。

评论 2.1.16. 从现在开始，我们在记法上会更加懒惰，假设读者能够从我们陈述的书写方式中推导出某些条件。例如，我们可能以“设 $B : A \to \mathcal{U}$ 是一个类型族”开始一个陈述。在这种情况下，读者应假设我们还要求 $A : \mathcal{U}$ 而没有其他限制。我们只在未指定变量只有“一个合理选择”时使用这种写法。应始终假设它具有上下文所需的最小能力。

由于我们广泛使用类型族，我们为它们引入有用的记法和概念：

定义 2.1.17. 对于类型族 $B : A \to \mathcal{U}$，我们定义：

- B 的全类型是 $\Sigma(a : A, B(a))$。我们也写作 ΣB。
- 全类型的投影是函数 $\pi_B : \Sigma B \to A, \quad (a, b) \mapsto a$。
- B 的依赖函数类型是 $(a : A) \to B(a)$。我们也写作 ΠB。这也称为 B 的截面，我们将在第 5 节中介绍和解释。

我们现在可以定义一个单一的恒等函数：

定义 2.1.18 (恒等函数)**.** 恒等函数 id 定义如下：

$$\mathrm{id} : (A : \mathcal{U}, a : A) \to A \qquad \mathrm{id} := \lambda A : \mathcal{U}.\lambda a : A.a$$

我们仍然继续写和使用 $\mathrm{id}_A : A \to A$，意思是上述定义下的 $\mathrm{id}(A)$。

评论 2.1.19. 在 rzk 中，我们实际上没有办法指定依赖类型，因为蕴涵不存在。相反，我们总是给出类型族。将变量放在定义的上下文中还是使其成为定义内部 Π-类型的参数之间也没有真正的区别。因此我们之前这样定义的恒等函数：

```
#def id (A : U) (a : A) : A := a
```

可以等价地定义为：

```
#def id (A : U) : A -> A := \a -> a
```

2.1.11 同伦

在完成了 Martin-Löf 类型论中最重要的概念之后，我们现在介绍 HoTT 中最重要的概念：命题相等，也称为恒等类型、路径类型或简称为 =-类型。我们仍然遵循 [1] 和 [19]。正如我们在介绍判断相等时已经讨论过的，这两个概念是不同的。判断相等是一个判断，所以它要么成立要么不成立；而命题相等是一个类型，所以我们在理论中有类型 $x =_A y$ 的一个元素，它证明了 x 等于 y（在类型 A 中）这一事实。然而，这个证明不是“要么存在要么不存在”的东西，而是可能附加了一些额外的数据。它存储了 x 等于 y 的方式，并且可能存在多种不同的方式使得两个值相等。

这给我们带来了一个重要观察：我们可以嵌套命题相等类型，本质上是问两个证明 $p, q : x =_A y$ 本身是否相等，即 $p =_{x=y} q$ 是否居留。这个问题可以递归地提出，给出两个证明两个证明相等的证明。用等式写出来，这很快变得难以解析：

$$p, q : x =_A y \qquad H_1, H_2 : p =_{x=y} q \qquad H_1 =_{p=y} H_2$$

这是恒等类型无限结构的一瞥。人们可能担心这种记法的索引漂移。但由于我们总是需要指定元素，考虑等式的类型几乎总是从上下文中可知。此外，我们很少考虑等式类型的多重嵌套，而是以一种允许代入任何类型的通用形式来书写定理。

=-类型的主要替代解释是路径的解释：$p : x = y$ 是从 x 到 y 在 A 中的一条路径。这种解释当然采用了同伦论的观点。所有对同伦论有趣的属性都源于这个类型的消去规则。如果之前从未见过，它相当神秘，所以需要一些时间来适应。

定义 2.1.20 (=-类型)**.** 我们已经给出了 =-类型的语法。给定两个元素 $x, y : A$，我们写 $x =_A y$ 表示 x 和 y 之间路径或等式的类型。当 x 和 y 的类型从上下文中清晰可见时（通常如此），我们也只写 $x = y$。对于消去规则，我们需要一个特殊符号，我们使用 J。

形成和引入规则是直接的：

$$\frac{\Gamma \vdash A : \mathcal{U} \quad \Gamma \vdash x : A \quad \Gamma \vdash y : A}{\Gamma \vdash x =_A y : \mathcal{U}} \qquad \frac{\Gamma \vdash A : \mathcal{U} \quad \Gamma \vdash x : A}{\Gamma \vdash \mathrm{refl}_x : x =_A x}$$

如果从上下文中不明显，我们也允许在等号下方书写类型 $x =_A y$；对于值也一样，所以 $\mathrm{refl}_x : x = x$。消去规则如前所述相当复杂，我们在下面给出更广泛的解释：

$$\frac{\Gamma, x : A, y : A, p : x =_A y \vdash C : \mathcal{U} \quad \Gamma, x : A \vdash c : C[x, y, p \mapsto x, x, \mathrm{refl}_x] \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A \quad \Gamma \vdash q : a =_A b}{\Gamma \vdash J(C, c, a, b, q) : C[a, b, q]}$$

我们与 $\equiv$ 的兼容性由以下规则给出：

$$J(C)(x, \mathrm{refl}_x) \equiv c$$

在消去规则中，我们从任何类型 A 开始，然后固定一个起点 $x : A$。接下来，我们有一个类型族 C，它允许依赖于 A 中一条以 x 为起点的路径，但有一个自由的终点 y。作为规则的最后一个要素，我们有一个证明 c，表明类型族 C 对于终点 x 和路径 refl_x 是居留的。消去规则的结论是，对于任何以 x 为起点的路径，C 都是居留的。这个规则也称为（基点）路径归纳法，因为证明 refl_x 的（依赖）陈述 C 就能证明所有其他路径 $p : x = y$ 的 C。

消去规则的另一个视角需要我们稍微向前跳跃，使用我们尚未引入的等价概念。消去规则允许我们的理论在等价下保持不变。考虑以下交换图：

$$\begin{array}{ccc} (x : A) \to C(x, x, \mathrm{refl}_x) & \xrightarrow{\sim} & (x : A, y : A, p : x = y) \to C(x, y, p) \\ \downarrow & & \downarrow \\ C(x, x, \mathrm{refl}_x) & \to & C(x, y, p) \end{array}$$

我们稍后会证明底部的映射是等价。通过这个等价，我们应该能够获得右侧垂直映射的一个截面，如图中虚线映射所示。从这个角度来看，虚线映射就是 =-类型的消去规则，因为我们只需要证明交换三角形的存在性（一旦我们有了 c，它就存在）。当我们稍后讨论将等价与命题相等等同起来的幺半等价性公理时，这种解释将获得更多可信度。

注意，我们的消去规则表述有点不寻常，经典变体将函数反柯里化，并允许族随起点 x 变化：

$$A : \mathcal{U} \quad C : (x : A, y : A, p : x = y) \to \mathcal{U} \quad c : (a : A) \to C(a, a, \mathrm{refl}_a) \quad x : A \quad y : A \quad p : x = y$$

$$J(C,p):C(x,y,p)$$

虽然我们的基于路径归纳法的规则和这个非基于路径归纳法的规则之间存在细微差别（取决于我们将其指定为规则还是理论中的定理），但它们是等价的。这在 [1, 第 1.12.2 节] 中有所展示，我们在此不深入细节。我们注意到只能固定两个端点之一，因为如果我们能固定两个端点，我们就能得出任何 $p:x=y$ 都有 $p=\mathrm{refl}_x$ 的结论，这将使所有更高结构坍塌。

评论 2.1.21. 两个元素的判断相等蕴含命题相等。这是因为判断相等与依赖类型和项兼容。因此，给定 $x\equiv y$，我们有 $(x=y)\equiv(x=x)\equiv(y=y)$，并且 $\mathrm{refl}_x:x=y$ 以及 $\mathrm{refl}_y:x=y$。我们还观察到两个 refl 是相同的：$\mathrm{refl}_x\equiv\mathrm{refl}_y$。

评论 2.1.22. 在 rzk 中，语法相同，不指定类型时写 $x=y$，指定类型时写 $x=_A y$。这是唯一支持类型推断的类型。refl 也叫做 refl，并且支持推断项。如果需要指定，语法是 $\mathrm{refl}_{x:A}$，其中 A 是 x 的类型。

群胚结构

恒等类型表现出一个群胚结构，我们可以完全从消去规则推导出来。我们首先必须定义反转和连接路径的操作。这反过来证明 $x=y$ 是一个等价关系，以及 $\mathrm{refl}_x:x=x$ 这一事实。

定理 2.1.23 (= 是等价关系)**.** *给定类型 $A:\mathcal{U}$ 及其三个元素 $x,y,z:A$，存在以下两个函数：*

$$(-)^{-1}:(x=y)\to(y=x)\qquad -\bullet-:(x=y)\to(y=z)\to(x=z)$$

第一个称为路径反转，第二个称为路径连接。

证明. 我们使用恒等类型的消去规则构造这两个函数。在第一种情况，我们将其应用于输入参数，因此我们需要证明存在一个 $x=x$ 的元素（因为通过路径归纳法，y 现在已被 x 替换）。但这当然是可能的，因为 refl_x 总是存在。

构造第二个函数类似，我们只需要应用两次路径归纳法，此时 $z\equiv y\equiv x$，我们再次可以选择 refl_x。 □

这两个函数满足群胚律，这在 [1, 第 2.1 节] 中得到了证明。我们在这里引用他们的结果：

定理 2.1.24 ($x=y$ 形成一个群胚)**.** *在任何类型 $A:\mathcal{U}$ 中，对于 $x,y,z,w:A$ 和 $p:x=y,q:y=z,r:z=w$，我们有：*

$$p\bullet p^{-1}=\mathrm{refl}_x\quad p^{-1}\bullet p=\mathrm{refl}_y\quad (p^{-1})^{-1}=p\quad \mathrm{refl}_x^{-1}\equiv\mathrm{refl}_x$$

$$p\bullet\mathrm{refl}_y=p\quad \mathrm{refl}_x\bullet p=p\quad (p\bullet q)\bullet r=p\bullet(q\bullet r)\quad \mathrm{refl}_x\bullet\mathrm{refl}_x\equiv\mathrm{refl}_x$$

由于所有这些都只在命题上成立（除了关于 refl 的规则），所有这些结构仅“直到更高阶一致性”成立。这正是 ∞-群胚所期望的。因为我们可以选择 A 本身是一个恒等类型，我们也得到了递归恒等类型的所有这些结构。

在理论上定义了所有这些之后，我们给出 S^1 的例子。

例 2.1.25 (圆)**.** 为了定义圆，我们使用 [1, 第 6 章] 中描述的生成元和关系。类型 S^1 由一个名为 $*$ 的单点（如果有歧义则为 $*_{S^1}$）和一条名为 $\mathsf{loop} : * = *$ 的路径组成。

生成元和关系机制先验地不会产生任何更高阶的一致性，因此 $\mathsf{loop} \neq \mathsf{loop} \bullet \mathsf{loop}$ 且 $\mathsf{loop} \neq \mathsf{loop}^{-1}$。事实上，[1] 明确证明了 $\mathsf{loop} \neq \mathrm{refl}$。此外，它没有添加 $\mathbb{1}$ 的消去规则（证明当 $x : \mathbb{1}$ 时 $*_{\mathbb{1}} \equiv x$ 的规则），因为那将损害 $*_{S^1}$ 的结构。这意味着虽然 S^1 有一个典范元素 $*_{S^1}$，但我们不能证明它是唯一的元素。

这看起来可能令人困惑，但它完美地符合 S^1 的同伦论观点。它由一个连通分支组成，所以所有的点都是等价的，但我们不能假设一开始只有一个点。[1] 证明了 $(*_{S^1} = *_{S^1}) \simeq \mathbb{Z}$，正如人们所期望的那样。

函子性

函数自动满足关于群胚结构的函子性。这一事实由以下定义记录：

定义 2.1.26 (函数在路径上的作用)**.** 设 $f : A \to B$ 是一个函数，$x, y : A$。我们使用路径归纳法定义以下函数：

$$\mathrm{ap}_f : (x = y) \to (f(x) = f(y))$$

它将 refl_x 发送到 $\mathrm{refl}_{f(x)}$。

我们还需要证明这个操作确实是函子性的：

定理 2.1.27. *（参见 [1, 引理 2.2.2]）对于函数 $f : A \to B, g : B \to C$ 和路径 $p : x =_A y, q : y =_A z$，以下陈述成立：*

$$\mathrm{ap}_f(p \bullet q) = \mathrm{ap}_f(p) \bullet \mathrm{ap}_f(q) \qquad \mathrm{ap}_f(p^{-1}) = \mathrm{ap}_f(p)^{-1}$$

$$\mathrm{ap}_g(\mathrm{ap}_f(p)) = \mathrm{ap}_{g \circ f}(p) \qquad \mathrm{ap}_{\mathrm{id}_A}(p) = p$$

2.1.12　可缩性

在本节的剩余部分，我们介绍同伦类型论中将在论文后面使用的重要术语。我们从可缩性的概念开始。

定义 2.1.28 (可缩)**.** 称类型 $A : \mathcal{U}$ 是可缩的，如果收缩的类型是居留的：

$$\mathsf{is\text{-}contr}(A) := \Sigma(x : A, (a : A) \to (a = x))$$

第一个分量 x 称为收缩的中心，第二个分量称为收缩的同伦。

一个可缩类型本质上只包含一个元素。起初这可能看起来这个定义只确保了恒等类型的第一层坍塌，而更高结构仍然是允许的。然而，情况并非如此。我们的 S^1 的例子不是可缩的。这起初可能看起来奇怪，但这是由于我们之前在本节中描述的 Π-类型的连续性。为了证明 S^1 不能被证明是可缩的，我们给出以下引理，它也为我们定义的可缩性名称提供了理由：

引理 2.1.29. *如果 $A : \mathcal{U}$ 是可缩的，那么 A 中的任何恒等类型也是可缩的。*

证明. 设 a 是 A 的收缩中心。我们记 $p_x : a = x$ 为从该收缩到任何其他点 $x : A$ 的同伦。

我们必须证明 $(x : A, y : A) \to \mathsf{is\text{-}contr}(x = y)$ 成立。固定 $x, y : A$ 并定义一个收缩：作为中心，我们选择 $p_x^{-1} \bullet p_y : x = y$。对于同伦，固定 $q : x = y$ 并对 q 应用路径归纳法。我们观察到现在的中心是 $\mathrm{refl} : p_x^{-1} \bullet p_x \equiv \mathrm{refl}_x$，因此我们可以使用 $\mathrm{refl}_{\mathrm{refl}_x}$ 作为中心和 $q \equiv \mathrm{refl}_x$ 之间的等式。 □

一个非常重要的陈述，我们将在后面广泛使用，是固定起点的路径类型是可缩的。这直接从消去规则得出。

引理 2.1.30. 对于任何类型 A 和任何值 $a : A$，$\Sigma(x : A, a = x)$ 是可缩的，其中心为 (a, refl_a)。

2.1.13　命题

定义 2.1.31. 称类型 $A : \mathcal{U}$ 是一个命题，如果任意两个居留者典范地相等：

$$\mathsf{is\text{-}prop}(A) := (x : A, y : A) \to \mathsf{is\text{-}contr}(x = y)$$

我们还将“命题宇宙”定义为 $\mathbb{P} := \Sigma(A : \mathcal{U}, \mathsf{is\text{-}prop}(A))$，并允许将此类型的元素隐式地强制转换为其第一个分量。

如果一个类型是命题，那就意味着该类型所代表的陈述为真的方式只有一种。用证明论的话说，一个命题的任何两个证明都是等价的。有几种替代方式来定义命题的概念：作为一个所有元素都相等的类型 $(x : A, y : A) \to x = y$，或者作为一个如果居留则可缩的类型 $A \to \mathsf{is\text{-}contr}(A)$。

由于 $\to$ 对应于逻辑蕴含，后一种定义对于获得命题概念的直观理解尤其有意义。一个类型是命题，如果它要么非居留（所以没有它的证明），要么是可缩的。前一种定义在想要证明某个类型是命题时非常有用。

我们注意到，如果为命题引入排中律，我们可以从命题中恢复经典逻辑。稍后讨论幺半等价性公理时，我们会观察到我们不能对任意类型这样做，因为这些公理相互冲突。

对于一个给定的命题 A，我们说“A 成立”意味着 A 是居留的。在 rzk 和一般类型论中的惯例是，只有当定义是一个命题时，才给它起名为“is-xyz”。我们的“is-contr”和“is-prop”定义就是这种情况。我们通常不会明确证明这一事实。在类型不立即显然是命题的情况下，[3] 通常提供证明。

值得注意的是，成为命题在我们几个类型构造子下是封闭的：

定理 2.1.32 (命题的封闭性质)**.** 给定一个类型族 $B : A \to \mathbb{P}$，其纤维-*wise* 是命题（这由其陪域 $\mathbb{P}$ 所要求），那么：

- 关联的依赖函数类型 $\Pi B \equiv (a : A) \to B(a)$ 也是一个命题。
- 如果此外 A 是一个命题，那么全类型 $\Sigma B \equiv \Sigma(a : A, B(a))$ 也是一个命题。

另外，任何可缩类型都是一个命题。

2.1.14 等价

等价的概念允许我们以另一种不同于 $=$ 和 $\equiv$ 的概念来等同类型。它是三者中最弱的。范畴论者对这个概念非常熟悉（在高阶范畴中通常称为弱等价），因为它允许等同包含数量悬殊的元素的类型。决定因素是所比较类型的恒等类型。两个类型的连通分支是双射的，并且更高结构本身直到等价都是被认同的。

定义 2.1.33 (等价)**.** 给定两个类型 $A, B : \mathcal{U}$ 和一个映射 $f : A \to B$，我们说 f 是一个等价，如果以下命题成立：

$$\mathsf{is\text{-}equiv}(f) := \Sigma(g_1 : B \to A, (b : B) \to f(g(b)) = b) \times \Sigma(g_2 : B \to A, (a : A) \to g(f(a)) = a)$$

g_1 称为 f 的截面，g_2 称为 f 的收缩。我们也将 $f : A \simeq B$ 写作从 A 到 B 的等价的类型。

这个定义确实是一个命题并不是从定义中直接可见的。而使用单个函数作为逆的朴素定义并不是一个命题。然而，存在一个从朴素定义到正确定义的映射，它只是两次使用逆。

恒等函数平凡地是一个等价，使用 refl 作为同伦值。此外，类型 A 和 B 之间的任何路径 $p : A = B$ 通过路径归纳法产生一个等价 $A \simeq B$。我们也可以复合等价：

定理 2.1.34. （参见 *[1, 引理 2.4.12]*）设 $f : A \to B$ 和 $g : B \to C$ 是等价。它们的复合也是一个等价（通过复合它们各自的截面/收缩和同伦）。

获得等价的另一种方式是证明一个映射有可缩的纤维：

定理 2.1.35. （参见 *[1, 定理 4.4.5]*）函数 $f : A \to B$ 是一个等价当且仅当对每个 $b : B$，$\mathrm{fib}_f(b)$ 是可缩的。

等价满足通常的 2-3 性质：

定理 2.1.36. （参见 *[1, 定理 4.7.1]*）给定 $f : A \to B, g : B \to C$。如果 f, g 和 $g \circ f$ 中的任意两个是等价，那么第三个也是。

命题之间的等价只需要给出两个方向的映射：

定理 2.1.37. （参见 *[1, 引理 3.3.3]*）给定命题 $A, B : \mathbb{P}$ 和 $f : A \to B, g : B \to A$，那么 $A \simeq B$。

重要的等价

有几个重要的等价我们在后面会用到。此外，类型论中的许多公理都是用等价表述的。函数外延性公理指出逐点相等的函数在命题上相等。对于下一节中的“扩展类型”，添加了一个类似的公理。在本节的后面，我们将讨论幺半等价性，这也是一条声明等价的公理。

定义 2.1.38 (函数外延性)**.** （参见 [1, 公理 2.9.3]）对于任何类型 $A : \mathcal{U}$ 和任何两个依赖映射 $f, g : \Pi B$，其中 $B : A \to \mathcal{U}$，由路径归纳法定义的自然映射是一个等价：

$$(f = g) \to ((a : A) \to (f(a) = g(a)))$$

这条公理可以从我们未假定但稍后讨论的幺半等价性公理证明。我们大量使用这条公理，有时没有显式调用它。这是因为在 rzk 中，必须每文件引入该公理，并在每个依赖它的陈述中显式使用它。因此，哪些陈述需要它，哪些不需要，是没有混淆的。

我们接下来要介绍的等价是平凡可证的，在实践中经常被忽略，因为它们出现得太频繁。它们允许我们在函数中移动参数，交换 Σ-类型中的元素，并描述如何交换 Σ 和 Π 类型。第一个在经典数学中称为“选择公理”。在我们的理论中，它是一个我们可以证明的陈述。这是由于类型论的构造性本质。Σ 携带了一个证明以及性质的见证。由于这个原因，我们可以从存在性的逐点证明构造一个选择函数。[1] 还给出了一个更合理的类型论选择公理，它实际上是一条公理。出于这个原因，我们将以下称为“选择定理”。

Proposition 2.1.39 (选择定理)**.** *（参见 [1, 定理 2.15.7]）给定 $B : A \to \mathcal{U}$ 和 $C : (a : A, b : B(a)) \to \mathcal{U}$，以下映射是一个等价：*

$$((a : A) \to \Sigma(b : B(a), C(a, b))) \to \Sigma(b : (a : A) \to B(a), (a : A) \to C(a, b(a)))$$

$$F \mapsto (\pi_1 \circ F, \pi_2 \circ F)$$

Proposition 2.1.40 (Σ-类型的结合性)**.** *给定 $B : A \to \mathcal{U}$ 和 $C : \Sigma B \to \mathcal{U}$，以下两个类型之间的典范映射是一个等价：*

$$\Sigma C \equiv \Sigma(x : \Sigma B, C(x)) \equiv \Sigma(x : \Sigma(a : A, B(a)), C(x)) \simeq \Sigma(a : A, b : B(a), C(a, b))$$

Proposition 2.1.41 (Σ-类型的交换性)**.** *给定 $B, C : A \to \mathcal{U}$，以下两个类型之间的典范映射是一个等价：*

$$\Sigma(a : A, b : B(a), c : C(a)) \simeq \Sigma(a : A, c : C(a), b : B(a))$$

Proposition 2.1.42 (Π-类型的柯里化)**.** *给定 $B : A \to \mathcal{U}$ 和 $C : \Sigma B \to \mathcal{U}$，以下两个类型之间的典范映射是一个等价：*

$$((a : A, b : B(a)) \to C(a, b)) \simeq (x : \Sigma B) \to C(x)$$

在我们研究传输等价之前，我们给出一个有用的引理，用于证明两个依赖命题等价而不需要纤维-wise 等价：

引理 2.1.43. *给定两个类型 $A, A' : \mathcal{U}$ 以及其上的命题族 $B : A \to \mathbb{P}, B' : A' \to \mathbb{P}$。如果我们还有一个 $f : A \to A'$ 满足 $F : (a : A, b' : B'(f(a))) \to B(a)$，以及一个 $g : A' \to A$ 满足 $G : (a' : A', b : B(g(a'))) \to B'(a')$，那么 $\Pi B \simeq \Pi B'$。*

证明. 由于 ΠB 和 $\Pi B'$ 都是命题，给出来回的映射就足够了：

$$\Pi B \to \Pi B', \quad b \mapsto \lambda a'.G(a', b(g(a')))$$

以及反向映射。 □

传输

作为我们最后一个重要的等价，我们转向传输。这是一个等价，它将在基中连接纤维所基于的值之间的路径给定时，关联类型族的纤维。

定义 2.1.44. 给定一个类型族 $B : A \to \mathcal{U}$ 和基中的一条路径 $p : x =_A y$，我们将 p 的传输定义为映射 $tr_B^p : B(x) \to B(y)$，通过对 p 进行路径归纳法定义，将 refl_x 发送到 $\mathrm{id}_{B(x)}$。

传输满足许多有用的性质：它是一个等价，其逆是沿反向路径的传输；两个传输的复合是路径连接的传输；拉回变成与定义 2.2.7 的复合。有关细节，请参阅 [1, 引理 2.3.9-11]。

2.1.15　嵌入

嵌入是等价的一个弱化版本。嵌入不是要求连通分支上的双射以及兼容的群胚结构，而是仅在连通分支上是单射的，并且群胚结构兼容。

定义 2.1.45 (嵌入)**.** 函数 $f : A \to B$ 称为嵌入，如果以下成立：

$$\mathsf{is\text{-}emb}(f) := \mathsf{is\text{-}equiv}(\mathrm{ap}_f)$$

我们稍后将引入完全嵌入的概念，它对应于范畴论中的完全忠实函子。

2.1.16　幺半等价性

在本小节中，我们讨论幺半等价性公理。由于 E. Riehl 等人 [3] 不假定它，并且我们所有的证明都不需要它，我们不将其作为公理添加到我们的理论中。尽管如此，它是同伦类型论的一个组成部分，并且有人尝试将幺半等价性的有向版本 [24] 添加到类型论中。

该公理的陈述需要等价的概念，并且写下来相当简单：

定义 2.1.46 (幺半等价性)**.** 对于任意两个类型 $A, B : \mathcal{U}$，由路径归纳法定义的从 A 和 B 之间的等式到两者之间等价的典范映射是一个等价：

$$(A = B) \xrightarrow{\sim} (A \simeq B)$$

这条公理使我们不仅从直观的角度，而且直接从理论的形式主义中，将同构的对象视为相同！它表明所有的等价都源于 $\mathcal{U}$ 中的路径。

幺半等价性与经典排中律和双重否定律不相容。后者在 [1, 定理 3.2.2] 中展示，前者在 [1, 推论 3.2.7] 中展示。然而，我们被允许对命题假定这两条公理。但我们再次不需要诉诸于使用它们，所以我们将它们从我们的理论中省略。

2.1.17 Grothendieck 构造

现在我们讨论我们理论中笔直化和反笔直化的非常有趣的构造。对于本小节，我们需要假定幺半等价性公理。有了它，我们能够从高阶范畴论证明笔直化定理（尽管仅对群胚成立，因为我们的理论中还没有方向性）。

在我们讨论之前，我们定义几个尚不需要幺半等价性的概念和辅助陈述。

定义 2.1.47 (纤维)**.** 给定一个映射 $f : A \to B$，我们定义它的纤维：

$$\mathrm{fib}_f(b) := \Sigma(a : A, f(a) = b)$$

引理 2.1.48. *设 $B : A \to \mathcal{U}$ 是一个族，那么投影 π_B 在 a 处的纤维通过以下映射等价于族在 a 处的值：*

$$B(a) \to \mathrm{fib}_{\pi_B}(a), \quad b \mapsto ((a, b), \mathrm{refl}_a)$$

证明. 等价的逆由 $\lambda(x : \Sigma B, p : \pi_b(x) = a).tr^p_B(\pi_2(x))$ 给出。在 $B(a)$ 处的同伦是 refl_b，在 $\mathrm{fib}_{\pi_B}(a)$ 处的同伦通过对 p 的路径归纳法给出。 □

由于这个原因，我们也称 $B(a)$ 为族 B 的纤维。

引理 2.1.49. *设 $f : A \to B$ 是任意映射，那么以下映射是一个等价：*

$$\Sigma\,\mathrm{fib}_f \equiv \Sigma(b : B, \mathrm{fib}_f(b)) \to A, \quad (b, (a, p)) \mapsto a$$

Grothendieck 构造表明，类型 B 上的类型族等价于该类型 B 上的纤维化（其中纤维化只是类型映射）。为此，我们引入简写记号：

$$\mathsf{Fam}(B) := B \to \mathcal{U} \qquad \mathsf{Fib}(B) := \Sigma(A : \mathcal{U}, A \to B)$$

定理 2.1.50 (笔直化)**.** *（参见 [1, 定理 4.8.3]，[5, 定理 2.5.1]）对于任何类型 $B : \mathcal{U}$，（反）笔直化映射形成一个等价：*

$$\mathsf{Un}_B : \mathsf{Fam}(B) \to \mathsf{Fib}(B), \quad P \mapsto (\Sigma P, \pi_P) \qquad \mathsf{St}_B : \mathsf{Fib}(B) \to \mathsf{Fam}(B), \quad (A, f) \mapsto \mathrm{fib}_f$$

这个构造在不假定幺半等价性公理时仍然有意义；只是它不会是一个等价。尽管我们不假定它，我们仍然大量使用这个构造，因为它允许我们使用类型族来表述每个概念。从记法角度来看，类型族表现更好，因为它们通常给出其纤维化对应物的严格版本。严格意味着在纤维化对应物中出现的同伦可以选择为 refl。

评论 2.1.51. 这个 Grothendieck 构造是 ∞-群胚版本的一个忠实表示。我们稍后将讨论 ∞-范畴的相同构造，并注意到那里的故事并不成功。

作为本节的最后一个定义，我们有可复合的类型族：

定义 2.1.52. 给定一个类型族 $B : A \to \mathcal{U}$，以及第一个族之上的第二个族，即 $C : \Sigma B \to \mathcal{U}$，我们可以形成 B 和 C 的复合：

$$C \odot B : A \to \mathcal{U}, \quad a \mapsto \Sigma(b : B(a), C((a, b)))$$

这对应于应用 Grothendieck 构造时的普通函数复合。

在下一节中，我们将转向单纯类型论，它允许我们讨论 ∞-范畴而不仅仅是 ∞-群胚。

2.2　单纯 HoTT 与范畴论

在上一节中，我们介绍了同伦类型论及其核心概念。现在我们想为我们的理论添加方向性。为此，我们研究 E. Riehl 和 M. Shulman [2] 的单纯同伦类型论。这是一个多层类型论，每一层只能依赖于自身或前面的层。共有三层，前两层是立方体和拓扑。这些用于定义形状，然后形状可用于映射到经典类型中。

例如，类型 A 中的箭头类型 $\mathrm{arr}(A)$ 被定义为将一维立方体映射到 A 中。这个理论中的方向性来自一维立方体，它由两个元素 $0, 1$ 组成，且具有序关系 $0 < 1$。

正如我们在引言中已经提到的，这种方法允许我们保留所有 Π-类型，其缺点是我们有一些不是范畴的类型。因此，我们必须在我们的理论中引入范畴的内部概念，即 Rezk 类型的概念。

在本节中，我们介绍几个范畴论概念。从展示具有对象、态射及其复合的通常范畴概念开始。然后我们定义同构、始对象和终对象、完全忠实函子和伴随。此外，我们分析正方形如何分解为三角形，以及它们如何与自然变换相关联。

最后，我们观察到宇宙 $\mathcal{U}$ 不是范畴的范畴。这在对象层面上就已经失败了，对象是类型，不一定是 Rezk。最重要的是，Cat 的任何定义都不会是 Segal 的，这使得 Cat 中的态射复合不可能。此外，Grothendieck 构造也没有被忠实表示。

虽然我们给出了理论的许多推理规则，但我们不会穷尽，而是选择更简洁但可能有歧义的记法。特别地，我们从规则中省略了 A **type** 判断和其他分散注意力的记法。我们这样做是为了以更简单的方式更快地介绍理论，而不让难以理解的 formalism 迷惑读者。如果出现任何问题，我们请读者参考 E. Riehl 和 M. Shulman [2] 或 E. Riehl 等人 [3] 的形式化。由于我们的工作是使用 rzk [4] 形式化的，我们在更宽松的表述上没有任何形式上的问题。

2.2.1　立方体、拓扑和形状

该理论由三层组成：立方体、拓扑和类型。类型层与我们之前讨论的相同，但有一个新增内容：扩展类型。它们使用我们通过前两层定义的形状。

每一层指定一个上下文，我们在所有层中使用相同的蕴涵符号 $\vdash$，并像往常一样在左侧收集上下文。为了分隔上下文，我们使用竖线。我们总是使用 Ξ 表示立方体层的上下文，Φ 表示拓扑层的上下文，Γ 表示类型层的上下文。因此，使用所有上下文的带蕴涵的判断看起来像这样：$\Xi \mid \Phi \mid \Gamma \vdash \mathbb{1}$ **type**。我们在可能的情况下省略这个上下文，并更积极地使用局部规则形式。立方体上下文在规则的多数判断中通常是相同的，所以我们通常省略它。

立方体可以视为 $\mathbb{R}^n$ 中立方体的综合等价物。拓扑本质上只是布尔值，依赖拓扑则给我们一种从立方体中选择顶点、边、面和高维结构的方式。我们称这些依赖拓扑为形状。我们可以形成的四个基本形状的例子如图 1 所示。我们称第一个为 Δ^1，第二个为 Λ^2_0，第三个为 Δ^2，第四个为 Δ^3。

拓扑层

拓扑层由拓扑组成，它们本质上是“依赖于立方体的布尔值”，行为类似于“严格命题”。有假和真拓扑的常量，具有适当的消去和引入规则（真总是可以被证明，从假可以推导出任何东西）。此外，我们有合取和析取。重要的是，我们没有否定或蕴含。在这一层中，我们还添加了立方体的判断相等，这将是一个拓扑，以便下一层可以将其作为原始概念使用。

拓扑上下文使用字母 Φ。从现在开始，当我们以局部规则形式书写规则时，需要注意推断正确的上下文。例如，拓扑声明 χ tope 只允许有立方体上下文，而拓扑析取 $\varphi \vee \psi$ 既有立方体上下文又有拓扑上下文。

定义 2.2.1 (拓扑层)**.** 拓扑层包含比立方体层更多的构造。它包括（依赖于立方体的）析取、合取和相等。我们从简单的常量项开始：

$$\frac{}{\top \ \mathsf{tope}} \qquad \frac{}{\bot \ \mathsf{tope}} \qquad \frac{\Phi \vdash \varphi \ \mathsf{tope}}{\Phi \vdash \neg\varphi \ \mathsf{tope}}$$

接下来我们定义拓扑合取：

$$\frac{\Phi \vdash \varphi \ \mathsf{tope} \quad \Phi \vdash \psi \ \mathsf{tope}}{\Phi \vdash (\varphi \wedge \psi) \ \mathsf{tope}} \qquad \frac{\Phi \vdash \varphi \quad \Phi \vdash \psi}{\Phi \vdash \varphi \wedge \psi} \qquad \frac{\Phi \vdash \varphi \wedge \psi}{\Phi \vdash \varphi} \qquad \frac{\Phi \vdash \varphi \wedge \psi}{\Phi \vdash \psi}$$

以及析取：

$$\frac{\Phi \vdash \varphi \ \mathsf{tope} \quad \Phi \vdash \psi \ \mathsf{tope}}{\Phi \vdash (\varphi \vee \psi) \ \mathsf{tope}} \qquad \frac{\Phi \vdash \varphi}{\Phi \vdash \varphi \vee \psi} \qquad \frac{\Phi \vdash \psi}{\Phi \vdash \varphi \vee \psi}$$

$$\frac{\Phi, \varphi \vdash \chi \quad \Phi, \psi \vdash \chi \quad \Phi \vdash \varphi \vee \psi}{\Phi \vdash \chi}$$

我们还有一个用于立方体相等的拓扑。它本质上是立方体的判断相等。我们使用与类型层中判断相等相同的符号，因为我们只能比较立方体的元素或类型的元素，没有歧义的空间。它还包括乘积立方体、**1** 和 **2** 的 η 和 β 规则。

$$\frac{\Phi \vdash s : I \quad \Phi \vdash t : I}{\Phi \vdash (s \equiv t) \ \mathsf{tope}} \qquad \frac{\Phi \vdash s : I}{\Phi \vdash s \equiv s} \qquad \frac{\Phi \vdash s \equiv t}{\Phi \vdash t \equiv s} \qquad \frac{\Phi \vdash s \equiv t \quad \Phi \vdash t \equiv v}{\Phi \vdash s \equiv v}$$

$$\frac{\Phi \vdash t : \mathbf{1}}{\Phi \vdash t \equiv \star} \qquad \frac{}{\Phi \vdash 0 \equiv 1 \to \bot} \qquad \frac{\Phi \vdash s : I \quad \Phi \vdash t : J}{\Phi \vdash \pi_1((s,t)) \equiv s} \qquad \frac{\Phi \vdash s : I \quad \Phi \vdash t : J}{\Phi \vdash \pi_2((s,t)) \equiv t}$$

$$\frac{\Phi \vdash t : I \times J}{\Phi \vdash (\pi_1(t), \pi_2(t)) \equiv t} \qquad \frac{\Xi, x : I \vdash \psi \ \mathsf{tope} \quad \Xi \mid \Phi \vdash t \equiv s \quad \Xi \mid \Phi \vdash \psi[x \mapsto s]}{\Xi \mid \Phi \vdash \psi[x \mapsto t]}$$

除了相等，我们还需要通过一个比较拓扑给 **2** 一个序。我们以方程形式而不是推理规则给出这个比较拓扑的公理，因为它们更容易理解。在立方体上下文 $x : \mathbf{2}, y : \mathbf{2}$ 中，我们声明 $(x \leq y)$ tope，称为不等式拓扑。我们给出严格区间的公理：对于任何 $x, y, z : \mathbf{2}$，我们有：

$$(x \leq x) \quad (0 \leq x) \quad (x \leq 1) \quad (x \leq y), (y \leq z) \vdash (x \leq z) \quad (x \leq y), (y \leq x) \vdash (x \equiv y) \quad (x \leq y) \vee (y \leq x)$$

这些规则从左到右、从上到下称为：自反性、最小值、最大值、传递性、反对称性和完全性。

严格区间立方体 **2** 只声明 0 和 1 是它的元素。然而，公理并不禁止它们之间有更多的元素。事实上，这正是这个理论有趣的地方，因为该理论的一个有效模型使用 $[0,1] \subset \mathbb{R}$（或 $[0,1] \cap \mathbb{Q}$）作为区间。对 **2** 的直觉应该是一个连续区间。

类似于立方体，rzk 有一个称为 TOPE 的拓扑宇宙。一个形状现在是一个“函数” $\mathbf{I} \to \mathbf{TOPE}$，其中 $\mathbf{I}$ 是一个立方体。这也被解释为一个函数，但与下一层的 Π-类型没有任何关系。这些“拓扑函数”的参数本身也可以是拓扑函数。这就是我们如何定义子形状的方法：给定 $\Psi : \mathbf{I} \to \mathbf{TOPE}$，我们可以声明一个子形状 $\Phi : \Psi \to \mathbf{TOPE}$。常量 $\top, \bot$ 以及合取、析取、等式和不等式拓扑也都是可用的。

形状

正如我们之前解释的，形状是特定的依赖于立方体的拓扑，我们用来定义依赖于它们的项和类型。具体来说，形状是单例上下文中的一个拓扑。我们对形状使用类似集合的表示法。[2] 提到我们可以通过添加以下规则来形式化这个概念：

$$\frac{\Xi \vdash \psi \ \mathsf{tope}}{\Xi \mid \cdot \mid \cdot \vdash \{\psi\} \ \mathsf{shape}}$$

一类非常重要的形状是单纯形：

$$\Delta^n := \{(t_1, \ldots, t_n) : \mathbf{2}^n \mid t_n \leq \cdots \leq t_1\}$$

我们只需要 n 较小时的这种形状。

除了单纯形，我们定义了子形状的概念：我们写 $\phi \subset \psi$ 表示 ϕ 是 ψ 的子形状。形式上，子形状关系声明如下：

$$\frac{\Xi \vdash \phi \ \mathsf{tope} \quad \Xi \vdash \psi \ \mathsf{tope} \quad \Xi, \phi \vdash \psi}{\Xi \mid \cdot \mid \cdot \vdash \phi \subset \psi}$$

因此两个形状必须在相同的立方体上下文中声明，并且每当子形状挑选出一个点时，超形状也必须包含它。

正如人们所期望的，我们可以定义单纯形的内角和外角。由于我们只需要低维的，我们只给出最重要的一个：

$$\Lambda^2_1 := \{(t, s) : \mathbf{2}^2 \mid t \equiv 1 \vee s \equiv 0\}$$

另一个相当重要的子形状是 Δ^1 的边界：

$$\partial\Delta^1 := \{t : \mathbf{2} \mid t \equiv 0 \vee t \equiv 1\}$$

评论 2.2.2. U. Buchholtz 和 J. Weinberger [5] 将形状重新解释为类型，这在几个方面被证明是有利的。例如，人们能够定义形状之间的映射，然后可以用来定义类型上的映射。Λ^2_1 是 Δ^2 的一个子形状，包含映射的后复合

$$(\Delta^2 \to A) \to (\Lambda^2_1 \to A)$$

在我们后面讨论 Segal 类型时起着重要作用。在我们不允许将形状强制转换为类型的解释中，我们无法将这个映射谈论为后复合。相反，我们必须像这样定义它：

$$\lambda f : \Delta^2 \to A.\lambda t : \Lambda^2_1.f(t)$$

虽然这很容易定义，但还有其他更复杂的情况。我们保持形状和类型为不同概念的原因是因为 rzk 也有这种行为，我们无法形式化任何依赖于这种强制转换存在的概念。

形状运算

从基本的单纯形，我们可以通过以下形状运算形成更复杂的形状：并集、乘积和子形状乘积。

定义 2.2.3. 给定相同立方体上的两个形状 $\{t : I \mid \psi\}$ 和 $\{t : I \mid \zeta\}$，我们可以形成它们的并集：

$$\psi \cup \zeta := \{t : I \mid \psi \vee \zeta\}$$

从两个形状，我们可以通过在其立方体的乘积上给出一个新形状来形成它们的乘积。

定义 2.2.4. 给定立方体 I 和 J 上的形状 $\{t : I \mid \psi\}$ 和 $\{s : J \mid \zeta\}$，我们将它们的乘积定义为：

$$\psi \times \zeta := \{(t, s) : I \times J \mid \psi \wedge \zeta\}$$

由于我们经常考虑子形状包含，我们还需要能够在超形状的乘积中形成两个子形状的相对乘积。这个相对乘积与两个子形状的实际乘积不同，因为当人们想在扩展类型的柯里化定理中使用它时，它没有正确的性质。

定义 2.2.5. 给定两个形状包含 $\phi \subset \{t : I \mid \psi\}$ 和 $\chi \subset \{s : J \mid \zeta\}$，我们定义 ϕ 和 χ 的推出积（我们也称之为子形状乘积）：

$$\phi \otimes \chi := (\phi \times \zeta) \cup (\psi \times \chi) \equiv \{(t, s) : I \times J \mid (\phi \wedge \zeta) \vee (\psi \wedge \chi)\}$$

例 2.2.6. 一个非常常用的乘积形状是 $\Delta^1 \times \Delta^1$。然而，它的拓扑条件并不有趣，因为它只是 $\top$。但是，如果我们考虑两种情况下的 $\partial\Delta^1 \subset \Delta^1$，我们得到一个有趣的子形状乘积：

$$\partial\Delta^1 \otimes \partial\Delta^1 \equiv \{(t, s) : \mathbf{2} \times \mathbf{2} \mid t \equiv 0 \vee t \equiv 1 \vee s \equiv 0 \vee s \equiv 1\}$$

这是正方形的完整边界。